CHAPTER 14

More about Classes

TOPICS

14.1 Instance and Static Members
14.2 Friends of Classes
14.3 Memberwise Assignment
14.4 Copy Constructors
14.5 Operator Overloading
14.6 Object Conversion
14.7 Aggregation
14.8 Focus on Object-Oriented Design: Class Collaborations
14.9 Focus on Object-Oriented Programming: Simulating the Game of Cho-Han
14.10 Rvalue References and Move Semantics

14.1 Instance and Static Members

CONCEPT: **Each instance of a class has its own copies of the class's instance variables. If a member variable is declared `static`, however, all instances of that class have access to that variable. If a member function is declared `static`, it may be called without any instances of the class being defined.**

Instance Variables

Each class object (an instance of a class) has its own copy of the class's member variables. An object's member variables are separate and distinct from the member variables of other objects of the same class. For example, recall that the `Rectangle` class discussed in Chapter 13 has two member variables: `width` and `length`. Suppose we define two objects of the `Rectangle` class and set their `width` and `length` member variables as shown in the following code.

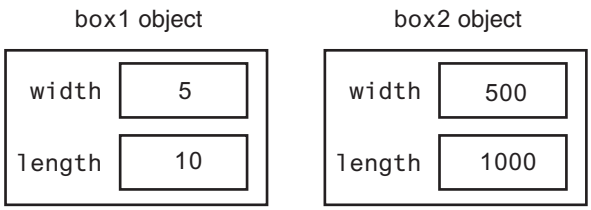
```
Rectangle box1, box2;

// Set the width and length for box1.
box1.setWidth(5);
box1.setLength(10);

// Set the width and length for box2.
box2.setWidth(500);
box2.setLength(1000);
```

This code creates box1 and box2, which are two distinct objects. Each has its own width and length member variables, as illustrated in Figure 14-1.

Figure 14-1 Two objects



When the getWidth member function is called, it returns the value stored in the calling object's width member variable. For example, the following statement displays 5 500.

```
cout << box1.getWidth() << " " << box2.getWidth() << endl;
```

In object-oriented programming, member variables such as the Rectangle class's width and length members are known as *instance variables*. They are called instance variables because each instance of the class has its own copies of the variables.

Static Members

It is possible to create a member variable or member function that does not belong to any instance of a class. Such members are known as *static member variables* and *static member functions*. When a value is stored in a static member variable, it is not stored in an instance of the class. In fact, an instance of the class doesn't even have to exist in order for values to be stored in the class's static member variables. Likewise, static member functions do not operate on instance variables. Instead, they can operate only on static member variables. You can think of static member variables and static member functions as belonging to the class instead of to an instance of the class. In this section, we will take a closer look at static members. First, we will examine static member variables.

Static Member Variables

When a member variable is declared with the key word static, there will be only one copy of the member variable in memory, regardless of the number of instances of the class that might exist. A single copy of a class's static member variable is shared by all instances of the class. For example, the following Tree class uses a static member variable to keep count of the number of instances of the class that are created.

Contents of Tree.h

```
1  // Tree class
2  class Tree
3  {
4  private:
5      static int objectCount;     // Static member variable.
6  public:
7      // Constructor
8      Tree()
9         { objectCount++; }
10
```

```
11      // Accessor function for objectCount
12      int getObjectCount() const
13         { return objectCount; }
14  };
15
16  // Definition of the static member variable, written
17  // outside the class.
18  int Tree::objectCount = 0;
```

First, notice in line 5 the declaration of the static member variable named `objectCount`: A static member variable is created by placing the key word `static` before the variable's data type. Also notice in line 18, we have written a definition statement for the `objectCount` variable, and that the statement is outside the class declaration. This external definition statement causes the variable to be created in memory, and is required. In line 18, we have explicitly initialized the `objectCount` variable with the value 0. We could have left out the initialization because C++ automatically stores 0 in all uninitialized static member variables. It is a good practice to initialize the variable anyway, so it is clear to anyone reading the code that the variable starts out with the value 0.

Next, look at the constructor in lines 8 and 9. In line 9, the `++` operator is used to increment `objectCount`. Each time an instance of the `Tree` class is created, the constructor will be called, and the `objectCount` member variable will be incremented. As a result, the `objectCount` member variable will contain the number of instances of the `Tree` class that have been created. The `getObjectCount` function, in lines 12 and 13, returns the value in `objectCount`. Program 14-1 demonstrates this class.

Program 14-1

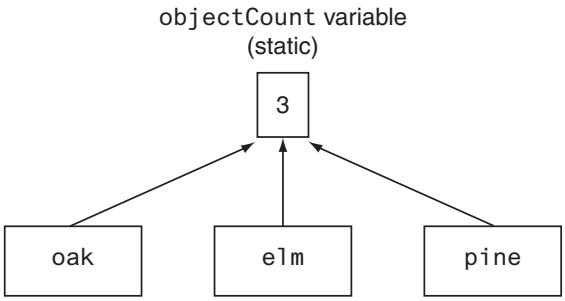
```
1  // This program demonstrates a static member variable.
2  #include <iostream>
3  #include "Tree.h"
4  using namespace std;
5
6  int main()
7  {
8      // Define three Tree objects.
9      Tree oak;
10     Tree elm;
11     Tree pine;
12
13     // Display the number of Tree objects we have.
14     cout << "We have " << pine.getObjectCount()
15          << " trees in our program!\n";
16     return 0;
17 }
```

Program Output

```
We have 3 trees in our program!
```

The program creates three instances of the `Tree` class, stored in the variables `oak`, `elm`, and `pine`. Although there are three instances of the class, there is only one copy of the static `objectCount` variable. This is illustrated in Figure 14-2.

Figure 14-2 A static variable



Instances of the Tree class

In line 14, the program calls the `getObjectCount` member function to retrieve the number of instances that have been created. Although the program uses the `pine` object to call the member function, the same value would be returned if any of the objects had been used. For example, all three of the following `cout` statements would display the same thing:

```
cout << "We have " << oak.getObjectCount() << " trees\n";
cout << "We have " << elm.getObjectCount() << " trees\n";
cout << "We have " << pine.getObjectCount() << " trees\n";
```

A more practical use of a static member variable is demonstrated in Program 14-2. The `Budget` class is used to gather the budget requests for all the divisions of a company. The class uses a static member, `corpBudget`, to hold the amount of the overall corporate budget. When the member function `addBudget` is called, its argument is added to the current contents of `corpBudget`. By the time the program is finished, `corpBudget` will contain the total of all the values placed there by all the `Budget` class objects. (These files can be found in the Student Source Code Folder Chapter 14\Budget Version 1.)

Contents of `Budget.h` (Version 1)

```
 1  #ifndef BUDGET_H
 2  #define BUDGET_H
 3
 4  // Budget class declaration
 5  class Budget
 6  {
 7  private:
 8      static double corpBudget;   // Static member
 9      double divisionBudget;      // Instance member
10  public:
11      Budget()
12          { divisionBudget = 0; }
13
14      void addBudget(double b)
15          { divisionBudget += b;
16            corpBudget += b; }
17
18      double getDivisionBudget() const
19          { return divisionBudget; }
20
```

```
21      double getCorpBudget() const
22        { return corpBudget; }
23  };
24
25  // Definition of static member variable corpBudget
26  double Budget::corpBudget = 0;
27
28  #endif
```

Program 14-2

```
1  // This program demonstrates a static class member variable.
2  #include <iostream>
3  #include <iomanip>
4  #include "Budget.h"
5  using namespace std;
6
7  int main()
8  {
9      int count;                          // Loop counter
10     const int NUM_DIVISIONS = 4;        // Number of divisions
11     Budget divisions[NUM_DIVISIONS];    // Array of Budget objects
12
13     // Get the budget requests for each division.
14     for (count = 0; count < NUM_DIVISIONS; count++)
15     {
16         double budgetAmount;
17         cout << "Enter the budget request for division ";
18         cout << (count + 1) << ": ";
19         cin >> budgetAmount;
20         divisions[count].addBudget(budgetAmount);
21     }
22
23     // Display the budget requests and the corporate budget.
24     cout << fixed << showpoint << setprecision(2);
25     cout << "\nHere are the division budget requests:\n";
26     for (count = 0; count < NUM_DIVISIONS; count++)
27     {
28         cout << "\tDivision " << (count + 1) << "\t$ ";
29         cout << divisions[count].getDivisionBudget() << endl;
30     }
31     cout << "\tTotal Budget Requests:\t$ ";
32     cout << divisions[0].getCorpBudget() << endl;
33
34     return 0;
35 }
```

Program Output with Example Input Shown in Bold

```
Enter the budget request for division 1: 100000 Enter
Enter the budget request for division 2: 200000 Enter
Enter the budget request for division 3: 300000 Enter
Enter the budget request for division 4: 400000 Enter
```

(program output continues)

Program 14-2 *(continued)*

```
Here are the division budget requests:
     Division 1     $ 100000.00
     Division 2     $ 200000.00
     Division 3     $ 300000.00
     Division 4     $ 400000.00
     Total Budget Requests:  $ 1000000.00
```

Static Member Functions

You declare a static member function by placing the `static` keyword in the function's prototype. Here is the general form:

```
static ReturnType FunctionName (ParameterTypeList);
```

A function that is a static member of a class cannot access any nonstatic member data in its class. With this limitation in mind, you might wonder what purpose static member functions serve. The following two points are important for understanding their usefulness:

1. Even though static member variables are declared in a class, they are actually defined outside the class declaration. The lifetime of a class's static member variable is the lifetime of the program. This means that a class's static member variables come into existence before any instances of the class are created.
2. A class's static member functions can be called before any instances of the class are created. This means that a class's static member functions can access the class's static member variables *before* any instances of the class are defined in memory. This gives you the ability to create very specialized setup routines for class objects.

Program 14-3, a modification of Program 14-2, demonstrates this feature. It asks the user to enter the main office's budget request before any division requests are entered. The `Budget` class has been modified to include a static member function named `mainOffice`. This function adds its argument to the static `corpBudget` variable, and is called before any instances of the `Budget` class are defined. (These files can be found in the Student Source Code Folder Chapter 14\Budget Version 2.)

Contents of `Budget.h` (Version 2)

```
 1  #ifndef BUDGET_H
 2  #define BUDGET_H
 3
 4  // Budget class declaration
 5  class Budget
 6  {
 7  private:
 8       static double corpBudget;   // Static member variable
 9       double divisionBudget;      // Instance member variable
10  public:
11       Budget()
12            { divisionBudget = 0; }
13
```

```
14      void addBudget(double b)
15           { divisionBudget += b;
16             corpBudget += b; }
17
18      double getDivisionBudget() const
19           { return divisionBudget; }
20
21      double getCorpBudget() const
22           { return corpBudget; }
23
24      static void mainOffice(double);  // Static member function
25  };
26
27  #endif
```

Contents of Budget.cpp

```
 1  #include "Budget.h"
 2
 3  // Definition of corpBudget static member variable
 4  double Budget::corpBudget = 0;
 5
 6  //*********************************************************
 7  // Definition of static member function mainOffice.       *
 8  // This function adds the main office's budget request to *
 9  // the corpBudget variable.                               *
10  //*********************************************************
11
12  void Budget::mainOffice(double moffice)
13  {
14      corpBudget += moffice;
15  }
```

Program 14-3

```
 1  // This program demonstrates a static member function.
 2  #include <iostream>
 3  #include <iomanip>
 4  #include "Budget.h"
 5  using namespace std;
 6
 7  int main()
 8  {
 9      int count;                    // Loop counter
10      double mainOfficeRequest;     // Main office budget request
11      const int NUM_DIVISIONS = 4;  // Number of divisions
12
13      // Get the main office's budget request.
14      // Note that no instances of the Budget class have been defined.
15      cout << "Enter the main office's budget request: ";
16      cin >> mainOfficeRequest;
17      Budget::mainOffice(mainOfficeRequest);
18
```

(program continues)

Program 14-3 *(continued)*

```
19      Budget divisions[NUM_DIVISIONS]; // An array of Budget objects.
20
21      // Get the budget requests for each division.
22      for (count = 0; count < NUM_DIVISIONS; count++)
23      {
24           double budgetAmount;
25           cout << "Enter the budget request for division ";
26           cout << (count + 1) << ": ";
27           cin >> budgetAmount;
28           divisions[count].addBudget(budgetAmount);
29      }
30
31      // Display the budget requests and the corporate budget.
32      cout << fixed << showpoint << setprecision(2);
33      cout << "\nHere are the division budget requests:\n";
34      for (count = 0; count < NUM_DIVISIONS; count++)
35      {
36           cout << "\tDivision " << (count + 1) << "\t$ ";
37           cout << divisions[count].getDivisionBudget() << endl;
38      }
39      cout << "\tTotal Budget Requests:\t$ ";
40      cout << divisions[0].getCorpBudget() << endl;
41
42      return 0;
43  }
```

Program Output with Example Input Shown in Bold

```
Enter the main office's budget request: 100000 Enter
Enter the budget request for division 1: 100000 Enter
Enter the budget request for division 2: 200000 Enter
Enter the budget request for division 3: 300000 Enter
Enter the budget request for division 4: 400000 Enter

Here are the division budget requests:
     Division 1     $ 100000.00
     Division 2     $ 200000.00
     Division 3     $ 300000.00
     Division 4     $ 400000.00
     Total Requests (including main office): $ 1100000.00
```

Notice in line 17 the statement that calls the static function `mainOffice`:

```
Budget::mainOffice(amount);
```

Calls to static member functions do not use the regular notation of connecting the function name to an object name with the dot operator. Instead, static member functions are called by connecting the function name to the class name with the scope resolution operator.



NOTE: If an instance of a class with a static member function exists, the static member function can be called with the class object name and the dot operator, just like any other member function.

14.2 Friends of Classes

CONCEPT: A friend is a function or class that is not a member of a class, but has access to the private members of the class.

Private members are hidden from all parts of the program outside the class, and accessing them requires a call to a public member function. Sometimes you will want to create an exception to that rule. A *friend* function is a function that is not part of a class, but that has access to the class's private members. In other words, a friend function is treated as if it were a member of the class. A friend function can be a regular stand-alone function, or it can be a member of another class. (In fact, an entire class can be declared a friend of another class.)

In order for a function or class to become a friend of another class, it must be declared as such by the class granting it access. Classes keep a "list" of their friends, and only the external functions or classes whose names appear in the list are granted access. A function is declared a friend by placing the key word `friend` in front of a prototype of the function. Here is the general format:

```
friend ReturnType FunctionName (ParameterTypeList)
```

In the following declaration of the `Budget` class, the `addBudget` function of another class, `AuxiliaryOffice` has been declared a friend. (This file can be found in the Student Source Code Folder Chapter 14\Budget Version 3.)

Contents of `Budget.h` (Version 3)

```
1  #ifndef BUDGET_H
2  #define BUDGET_H
3  #include "Auxil.h"
4
5  // Budget class declaration
6  class Budget
7  {
8  private:
9       static double corpBudget;   // Static member variable
10      double divisionBudget;      // Instance member variable
11 public:
12      Budget()
13           { divisionBudget = 0; }
14
15      void addBudget(double b)
16           { divisionBudget += b;
17             corpBudget += b; }
18
19      double getDivisionBudget() const
20           { return divisionBudget; }
21
22      double getCorpBudget() const
23           { return corpBudget; }
24
```

```
25      // Static member function
26      static void mainOffice(double);
27
28      // Friend function
29      friend void AuxiliaryOffice::addBudget(double, Budget &);
30  };
31
32  #endif
```

Let's assume another class, `AuxiliaryOffice`, represents a division's auxiliary office, perhaps in another country. The auxiliary office makes a separate budget request, which must be added to the overall corporate budget. The friend declaration of the `AuxiliaryOffice::addBudget` function tells the compiler that the function is to be granted access to `Budget`'s private members. Notice the function takes two arguments: a `double` and a reference object of the `Budget` class. The `Budget` class object that is to be modified by the function is passed to it, by reference, as an argument. The following code shows the declaration of the `AuxillaryOffice` class. (This file can be found in the Student Source Code Folder Chapter 14\Budget Version 3.)

Contents of `Auxil.h`

```
1   #ifndef AUXIL_H
2   #define AUXIL_H
3
4   class Budget; // Forward declaration of Budget class
5
6   // Aux class declaration
7
8   class AuxiliaryOffice
9   {
10  private:
11      double auxBudget;
12  public:
13      AuxiliaryOffice()
14          { auxBudget = 0; }
15
16      double getDivisionBudget() const
17          { return auxBudget; }
18
19      void addBudget(double, Budget &);
20  };
21
22  #endif
```

Contents of `Auxil.cpp`

```
1   #include "Auxil.h"
2   #include "Budget.h"
3
4   //*************************************************************
5   // Definition of member function mainOffice.                  *
6   // This function is declared a friend by the Budget class.    *
7   // It adds the value of argument b to the static corpBudget   *
8   // member variable of the Budget class.                       *
9   //*************************************************************
```

```
10
11  void AuxiliaryOffice::addBudget(double b, Budget &div)
12  {
13      auxBudget += b;
14      div.corpBudget += b;
15  }
```

Notice the Auxil.h file contains the following statement in line 4:

```
class Budget; // Forward declaration of Budget class
```

This is a *forward declaration* of the `Budget` class. It simply tells the compiler that a class named `Budget` will be declared later in the program. This is necessary because the compiler will process the Auxil.h file before it processes the `Budget` class declaration. When it is processing the Auxil.h file, it will see the following function declaration in line 19:

```
void addBudget(double, Budget &);
```

The `addBudget` function's second parameter is a `Budget` reference variable. At this point, the compiler has not processed the `Budget` class declaration, so, without the forward declaration, it wouldn't know what a `Budget` reference variable is.

The following code shows the definition of the `addBudget` function. (This file can be found in the Student Source Code Folder Chapter 14\Budget Version 3.)

Contents of `Auxil.cpp`

```
1  #include "Auxil.h"
2  #include "Budget.h"
3
4  //*************************************************************
5  // Definition of member function mainOffice.                  *
6  // This function is declared a friend by the Budget class.    *
7  // It adds the value of argument b to the static corpBudget   *
8  // member variable of the Budget class.                       *
9  //*************************************************************
10
11 void AuxiliaryOffice::addBudget(double b, Budget &div)
12 {
13     auxBudget += b;
14     div.corpBudget += b;
15 }
```

The parameter `div`, a reference to a `Budget` class object, is used in line 14. This statement adds the parameter `b` to `div.corpBudget`. Program 14-4 demonstrates the classes.

Program 14-4

```
1  // This program demonstrates a static member function.
2  #include <iostream>
3  #include <iomanip>
4  #include "Budget.h"
5  using namespace std;
6
```

(program continues)

Program 14-4 *(continued)*

```
 7  int main()
 8  {
 9      int count;                      // Loop counter
10      double mainOfficeRequest;       // Main office budget request
11      const int NUM_DIVISIONS = 4;    // Number of divisions
12
13      // Get the main office's budget request.
14      cout << "Enter the main office's budget request: ";
15      cin >> mainOfficeRequest;
16      Budget::mainOffice(mainOfficeRequest);
17
18      Budget divisions[NUM_DIVISIONS];  // Array of Budget objects
19      AuxiliaryOffice auxOffices[4];    // Array of AuxiliaryOffice
20
21      // Get the budget requests for each division
22      // and their auxiliary offices.
23      for (count = 0; count < NUM_DIVISIONS; count++)
24      {
25          double budgetAmount; // To hold input
26
27          // Get the request for the division office.
28          cout << "Enter the budget request for division ";
29          cout << (count + 1) << ": ";
30          cin >> budgetAmount;
31          divisions[count].addBudget(budgetAmount);
32
33          // Get the request for the auxiliary office.
34          cout << "Enter the budget request for that division's\n";
35          cout << "auxiliary office: ";
36          cin >> budgetAmount;
37          auxOffices[count].addBudget(budgetAmount, divisions[count]);
38      }
39
40      // Display the budget requests and the corporate budget.
41      cout << fixed << showpoint << setprecision(2);
42      cout << "\nHere are the division budget requests:\n";
43      for (count = 0; count < NUM_DIVISIONS; count++)
44      {
45          cout << "\tDivision " << (count + 1) << "\t\t$";
46          cout << divisions[count].getDivisionBudget() << endl;
47          cout << "\tAuxiliary office:\t$";
48          cout << auxOffices[count].getDivisionBudget() << endl << endl;
49      }
50      cout << "Total Budget Requests:\t$ ";
51      cout << divisions[0].getCorpBudget() << endl;
52      return 0;
53  }
```

Program Output with Example Input Shown in Bold

```
Enter the main office's budget request: 100000 Enter
Enter the budget request for division 1: 100000 Enter
Enter the budget request for that division's
auxiliary office: 50000 Enter
Enter the budget request for division 2: 200000 Enter
Enter the budget request for that division's
auxiliary office: 40000 Enter
Enter the budget request for division 3: 300000 Enter
Enter the budget request for that division's
auxiliary office: 70000 Enter
Enter the budget request for division 4: 400000 Enter
Enter the budget request for that division's
auxiliary office: 65000 Enter

Here are the division budget requests:
   Division 1                  $100000.00
   Auxiliary office:           $50000.00

   Division 2                  $200000.00
   Auxiliary office:           $40000.00

   Division 3                  $300000.00
   Auxiliary office:           $70000.00

   Division 4                  $400000.00
   Auxiliary office:           $65000.00

Total Budget Requests:  $ 1325000.00
```

As mentioned before, it is possible to make an entire class a friend of another class. The `Budget` class could make the `AuxiliaryOffice` class its friend with the following declaration:

```
friend class AuxiliaryOffice;
```

This may not be a good idea, however. Every member function of `AuxiliaryOffice` (including ones that may be added later) would have access to the private members of `Budget`. The best practice is to declare as friends only those functions that must have access to the private members of the class.



Checkpoint

14.1 What is the difference between an instance member variable and a static member variable?

14.2 Static member variables are declared inside the class declaration. Where do you write the definition statement for a static member variable?

14.3 Does a static member variable come into existence in memory before, at the same time as, or after any instances of its class?

14.4 What limitation does a static member function have?

14.5 What action is possible with a static member function that isn't possible with an instance member function?

14.6 If class `X` declares function `f` as a friend, does function `f` become a member of class `X`?

14.7 Class `Y` is a friend of class `X`, which means the member functions of class `Y` have access to the private members of class `X`. Does the friend key word appear in class `Y`'s declaration or in class `X`'s declaration?

14.3 Memberwise Assignment

CONCEPT: **The = operator may be used to assign one object's data to another object, or to initialize one object with another object's data. By default, each member of one object is copied to its counterpart in the other object.**

Like other variables (except arrays), objects may be assigned to one another using the = operator. As an example, consider Program 14-5, which uses the `Rectangle` class (version 4) we discussed in Chapter 13. Recall that the `Rectangle` class has two member variables: `width` and `length`. The constructor accepts two arguments: one for `width`, and one for `length`.

Program 14-5

```
 1  // This program demonstrates memberwise assignment.
 2  #include <iostream>
 3  #include "Rectangle.h"
 4  using namespace std;
 5
 6  int main()
 7  {
 8      // Define two Rectangle objects.
 9      Rectangle box1(10.0, 10.0);   // width = 10.0, length = 10.0
10      Rectangle box2 (20.0, 20.0);  // width = 20.0, length = 20.0
11
12      // Display each object's width and length.
13      cout << "box1's width and length: " << box1.getWidth()
14           << " " << box1.getLength() << endl;
15      cout << "box2's width and length: " << box2.getWidth()
16           << " " << box2.getLength() << endl << endl;
17
18      // Assign the members of box1 to box2.
19      box2 = box1;
20
21      // Display each object's width and length again.
22      cout << "box1's width and length: " << box1.getWidth()
23           << " " << box1.getLength() << endl;
24      cout << "box2's width and length: " << box2.getWidth()
25           << " " << box2.getLength() << endl;
26
27      return 0;
28  }
```

Program Output

```
box1's width and length: 10 10
box2's width and length: 20 20

box1's width and length: 10 10
box2's width and length: 10 10
```

The following statement, which appears in line 19, copies the `width` and `length` member variables of `box1` directly into the `width` and `length` member variables of `box2`:

```
box2 = box1;
```

Memberwise assignment also occurs when one object is initialized with another object's values. Remember the difference between assignment and initialization: assignment occurs between two objects that already exist, and initialization happens to an object being created. Consider the following code:

```
// Define box1.
Rectangle box1(100.0, 50.0);

// Define box2, initialize with box1's values
Rectangle box2 = box1;
```

The last statement defines a `Rectangle` object, `box2`, and initializes it to the values stored in `box1`. Because memberwise assignment takes place, the `box2` object will contain the exact same values as the `box1` object.

14.4 Copy Constructors

CONCEPT: **A copy constructor is a special constructor that is called whenever a new object is created and initialized with another object's data.**

Sometimes the default memberwise assignment behavior in C++ is perfectly acceptable. There are many situations, however, where memberwise assignment cannot be used. For example, consider the following class. (This file can be found in the Student Source Code Folder Chapter 14\StudentTestScores Version 1.)

Contents of `StudentTestScores.h` (Version 1)

```
 1  #ifndef STUDENTTESTSCORES_H
 2  #define STUDENTTESTSCORES_H
 3  #include <string>
 4  using namespace std;
 5
 6  const double DEFAULT_SCORE = 0.0;
 7
 8  class StudentTestScores
 9  {
10  private:
11      string studentName; // The student's name
12      double *testScores; // Points to array of test scores
13      int numTestScores;  // Number of test scores
14
15      // Private member function to create an
16      // array of test scores.
17      void createTestScoresArray(int size)
18      { numTestScores = size;
19        testScores = new double[size];
20        for (int i = 0; i < size; i++)
21            testScores[i] = DEFAULT_SCORE;}
22
```

```
23  public:
24       // Constructor
25       StudentTestScores(string name, int numScores)
26       { studentName = name;
27         createTestScoresArray(numScores); }
28
29       // Destructor
30       ~StudentTestScores()
31       { delete [] testScores; }
32
33       // The setTestScore function sets a specific
34       // test score's value.
35       void setTestScore(double score, int index)
36       { testScores[index] = score; }
37
38       // Set the student's name.
39       void setStudentName(string name)
40       { studentName = name; }
41
42       // Get the student's name.
43       string getStudentName() const
44       { return studentName; }
45
46       // Get the number of test scores.
47       int getNumTestScores() const
48       { return numTestScores; }
49
50       // Get a specific test score.
51       double getTestScore(int index) const
52       { return testScores[index]; }
53  };
54  #endif
```

This class stores a student's name and a set of test scores. Let's take a closer look at the code:

- Lines 11 through 13 declare the class's attributes. The `studentName` attribute is a `string` object that holds a student's name. The `testScores` attribute is an `int` pointer. Its purpose is to point to a dynamically allocated `int` array that holds the student's test score. The `numTestScore` attribute is an `int` that holds the number of test scores.
- The `createTestScoresArray` private member function, in lines 17 through 21, creates an array to hold the student's test scores. It accepts an argument for the number of test scores, assigns this value to the `numTestScores` attribute (line 18), then dynamically allocates an `int` array for the `testScores` attribute (line 19). The `for` loop in lines 20 and 21 initializes each element of the array to the default value 0.0.
- The constructor, in lines 25 through 27, accepts the student's name and the number of test scores as arguments. In line 26, the name is assigned to the `studentName` attribute, and in line 27 the number of test scores is passed to the `createTestScoresArray` member function.
- The destructor, in lines 30 and 31, deallocates the `testScores` array.
- The `setTestScore` member function, in lines 35 and 36, sets a specific score in the `testScores` attribute. The function accepts arguments for the score and the index where the score should be stored in the `testScores` array.

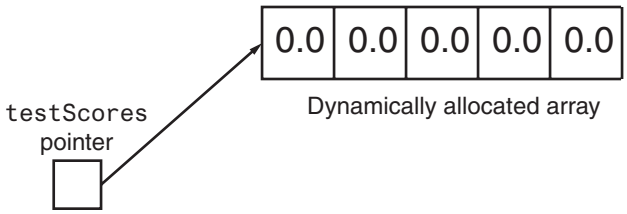
- The `setStudentName` member function, in lines 39 and 40, accepts an argument that is assigned to the `studentName` attribute.
- The `getStudentName` member function, in lines 43 and 44, returns the value of the `studentName` attribute.
- The `getNumTestScores` member function, in lines 47 and 48, returns the number of test scores stored in the object.
- The `getTestScore` member function, in lines 51 and 52, returns a specific score (specified by the index parameter) from the `testScores` attribute.

A potential problem with this class lies in the fact that one of its members, `testScores`, is a pointer. The `createTestScoresArray` member function (called by the constructor) performs a critical operation with the pointer: It dynamically allocates a section of memory for the `testScores` array and assigns default values to each of its element. For instance, the following statement creates a `StudentTestScores` object named `student1`, whose `testScores` member references dynamically allocated memory holding an array of 5 `double`'s:

```
StudentTestScores("Maria Jones Tucker", 5);
```

This is depicted in Figure 14-3.

Figure 14-3 The `testScores` member referencing a dynamically allocated array

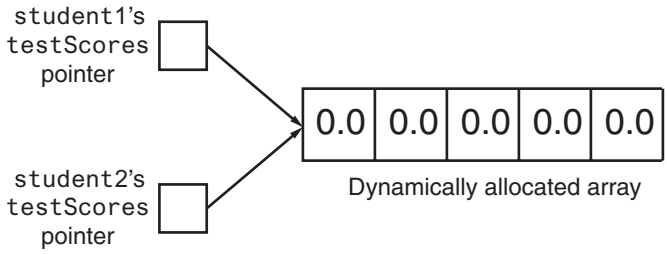


Consider what happens when another `StudentTestScores` object is created and initialized with the `student1` object, as in the following statement:

```
StudentTestScores student2 = student1;
```

In the statement above, `student2`'s constructor isn't called. Instead, memberwise assignment takes place, copying each of `student1`'s member variables into `student2`. This means that a separate section of memory is not allocated for `student2`'s `testScores` member. It simply gets a copy of the address stored in `student1`'s `testScores` member. Both pointers will point to the same address, as depicted in Figure 14-4.

Figure 14-4 Two pointers pointing to the same array



In this situation, either object can manipulate the values stored in the array, causing the changes to show up in the other object. Likewise, one object can be destroyed, causing its destructor to be called, which frees the allocated memory. The remaining object's `testScores` pointer would still reference this section of memory, although it should no longer be used.

The solution to this problem is to create a *copy constructor* for the object. A copy constructor is a special constructor that's called when an object is initialized with another object's data. It has the same form as other constructors, except it has a reference parameter of the same class type as the object itself. For example, here is a copy constructor for the `StudentTestScores` class:

```
StudentTestScores(StudentTestScores &obj)
{ studentName = obj.studentName;
  numTestScores = obj.numTestScores;
  testScores = new double[numTestScores];
  for (int i = 0; i < length; i++)
       testScores[i] = obj.testScores[i]; }
```

When the `=` operator is used to initialize a `StudentTestScores` object with the contents of another `StudentTestScores` object, the copy constructor is called. The `StudentTestScores` object that appears on the right side of the `=` operator is passed as an argument to the copy constructor. For example, look at the following statement:

```
StudentTestScores student1 ("Molly McBride", 8);
StudentTestScores student2 = student1;
```

In this code, the `student1` object is passed as an argument to the `student2` object's copy constructor.



NOTE: C++ requires that a copy constructor's parameter be a reference object.

As you can see from studying the copy constructor's code, `student2`'s `testScores` member will properly reference its own dynamically allocated memory. There will be no danger of `student1` inadvertently destroying or corrupting `student2`'s data.

Using `const` Parameters in Copy Constructors

Because copy constructors are required to use reference parameters, they have access to their argument's data. Since the purpose of a copy constructor is to make a copy of the argument, there is no reason the constructor should modify the argument's data. With this in mind, it's a good idea to make copy constructors' parameters constant by specifying the `const` key word in the parameter list. Here is an example:

```
StudentTestScores(const StudentTestScores &obj)
{ studentName = obj.studentName;
  numTestScores = obj.numTestScores;
  testScores = new double[numTestScores];
  for (int i = 0; i < numTestScores; i++)
       testScores[i] = obj.testScores[i]; }
```

The `const` key word ensures that the function cannot change the contents of the parameter. This will prevent you from inadvertently writing code that corrupts data.

The complete listing for the revised `StudentTestScores` class is shown here. (This file can be found in the Student Source Code Folder Chapter 14\StudentTestScores Version 2.)

Contents of `StudentTestScores.h` (Version 2)

```
#ifndef STUDENTTESTSCORES_H
#define STUDENTTESTSCORES_H
#include <string>
using namespace std;

const double DEFAULT_SCORE = 0.0;

class StudentTestScores
{
private:
    string studentName;  // The student's name
    double *testScores;  // Points to array of test scores
    int numTestScores;   // Number of test scores

    // Private member function to create an
    // array of test scores.
    void createTestScoresArray(int size)
    { numTestScores = size;
      testScores = new double[size];
      for (int i = 0; i < size; i++)
          testScores[i] = DEFAULT_SCORE; }

public:
    // Constructor
    StudentTestScores(string name, int numScores)
    { studentName = name;
      createTestScoresArray(numScores); }

    // Copy constructor
    StudentTestScores(const StudentTestScores &obj)
    { studentName = obj.studentName;
      numTestScores = obj.numTestScores;
      testScores = new double[numTestScores];
      for (int i = 0; i < numTestScores; i++)
          testScores[i] = obj.testScores[i]; }

    // Destructor
    ~StudentTestScores()
    { delete [] testScores; }

    // The setTestScore function sets a specific
    // test score's value.
    void setTestScore(double score, int index)
    { testScores[index] = score; }

```

```
46      // Set the student's name.
47      void setStudentName(string name)
48      { studentName = name; }
49
50      // Get the student's name.
51      string getStudentName() const
52      { return studentName; }
53
54      // Get the number of test scores.
55      int getNumTestScores() const
56      { return numTestScores; }
57
58      // Get a specific test score.
59      double getTestScore(int index) const
60      { return testScores[index]; }
61  };
62  #endif
```

Copy Constructors and Function Parameters

When a class object is passed by value as an argument to a function, it is passed to a parameter that is also a class object, and the copy constructor of the function's parameter is called. Remember that when a nonreference class object is used as a function parameter, it is created when the function is called, and it is initialized with the argument's value.

This is why C++ requires the parameter of a copy constructor to be a reference object. If an object were passed to the copy constructor by value, the copy constructor would create a copy of the argument and store it in the parameter object. When the parameter object is created, its copy constructor will be called, thus causing another parameter object to be created. This process will continue indefinitely (or at least until the available memory fills up, causing the program to halt).

To prevent the copy constructor from calling itself an infinite number of times, C++ requires its parameter to be a reference object.

The Default Copy Constructor

Although you may not realize it, you have seen the action of a copy constructor before. If a class doesn't have a copy constructor, C++ creates a *default copy constructor* for it. The default copy constructor performs the memberwise assignment discussed in the previous section.



Checkpoint

14.8 Briefly describe what is meant by memberwise assignment.

14.9 Describe two instances when memberwise assignment occurs.

14.10 Describe a situation in which memberwise assignment should not be used.

14.11 When is a copy constructor called?

14.12 How does the compiler know that a member function is a copy constructor?

14.13 What action is performed by a class's default copy constructor?

14.5 Operator Overloading

CONCEPT: **C++ allows you to redefine how standard operators work when used with class objects.**



VideoNote
Operator Overloading

C++ provides many operators to manipulate data of the primitive data types. However, what if you wish to use an operator to manipulate class objects? For example, assume that a class named `Date` exists, and objects of the `Date` class hold the month, day, and year in member variables. Suppose the `Date` class has a member function named `add`. The `add` member function adds a number of days to the date and adjusts the member variables if the date goes to another month or year. For example, the following statement adds 5 days to the date stored in the `today` object:

```
today.add(5);
```

Although it might be obvious that the statement is adding 5 days to the date stored in `today`, the use of an operator might be more intuitive. For example, look at the following statement:

```
today += 5;
```

This statement uses the standard `+=` operator to add 5 to `today`. This behavior does not happen automatically, however. The `+=` operator must be *overloaded* for this action to occur. In this section, you will learn to overload many of C++'s operators to perform specialized operations on class objects.



NOTE: You have already experienced the behavior of an overloaded operator. The / operator performs two types of division: floating-point and integer. If one of the / operator's operands is a floating-point type, the result will be a floating-point value. If both of the / operator's operands are integers, however, a different behavior occurs: the result is an integer, and any fractional part is thrown away.

The `this` Pointer

Before going any further with our discussion of operator overloading, we need to discuss a special pointer named `this`. The `this` pointer is a built-in pointer that every class has. It is passed as a hidden argument to all nonstatic member functions, and it always points to the instance of the class making the function call. For example, if `student1` and `student2` are both `StudentTestScores` objects, the following statement causes the `getStudentName` member function to operate on `student1`:

```
cout << student1.getStudentName() << endl;
```

Likewise, the following statement causes `getStudentName` to operate on `student2`:

```
cout << student2.getStudentName() << endl;
```

When `getStudentName` is operating on `student1`, the `this` pointer is pointing to `student1`. When `getStudentName` is operating on `student2`, the `this` pointer is pointing to `student2`. The `this` pointer always points to the object that is being used to call the member function.

Overloading the = Operator

Although copy constructors solve the initialization problems inherent with objects containing pointer members, they do not work with simple assignment statements. Copy constructors are just that—constructors. They are invoked only when an object is created. Statements like the following still perform memberwise assignment:

```
student2 = student1;
```

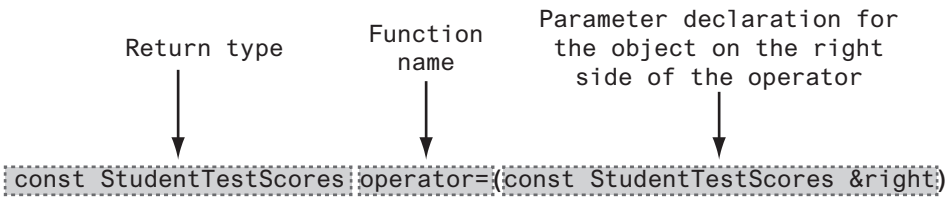
In order to change the way the assignment operator works, it must be overloaded. Operator overloading permits you to redefine an existing operator's behavior when used with a class object.

C++ allows a class to have special member functions called *operator functions*. If you wish to redefine the way a particular operator works with an object, you define a function for that operator. The operator function is then executed any time the operator is used with an object of that class. Let's look at how we might overload the = operator in the `StudentTestScores` class. If you go to this book's Student Source Code Folder named Chapter 14\StudentTestScores version 3 and open the StudentTestScores.h file, you will find that inside the class, in lines 63 through 74, we have added the following member function:

```
62  // Overloaded = operator
63  const StudentTestScores operator=(const StudentTestScores &right)
64  {  if (this != &right)
65     {
66        delete[] testScores;
67        studentName = right.studentName;
68        numTestScores = right.numTestScores;
69        testScores = new double[numTestScores];
70        for (int i = 0; i < numTestScores; i++)
71           testScores[i] = right.testScores[i];
72     }
73     return *this;
74  }
```

Let's examine the operator function to understand how it works. First, let's dissect the function header, in line 63, into the three parts shown in Figure 14-5.

Figure 14-5 Function header for `operator=`



- **Return type:** The function's return type is `const StudentTestScores`. This means that it returns a `StudentTestScores` object that is `const` (it cannot be modified). We will discuss this in greater detail in a moment.
- **Function name:** The function's name is `operator=`. This specifies that the function overloads the = operator. Because it is a member of the `StudentTestScores` class,

this function will be called when an assignment statement executes where the object on the left side of the = operator is a `StudentTestScores` object.

- **Parameter declaration:** The function has one parameter: a constant reference object named `right`. This parameter references the object on the right side of the operator. For example, when the following statement is executed, right will reference the `student1` object:

```
student2 = student1;
```

We declared the `right` parameter as a reference variable for efficiency purposes. This prevents the compiler from making a copy of the object that is being passed into the function. We declared it `const`, so the function will not accidentally change its contents.



NOTE: In the example, the parameter was named `right` simply to illustrate that it references the object on the right side of the operator. You can name the parameter anything you wish. It will always take the object on the operator's right as its argument.

In learning the mechanics of operator overloading, it is helpful to know that the following two statements do the same thing:

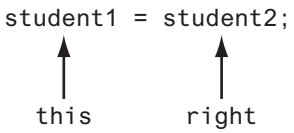
```
student2 = student1;           // Call operator= function
student2.operator=(student1); // Call operator= function
```

In the second statement, you can see exactly what is going on in the function call. The `student1` object is being passed to the function's parameter, `right`. Inside the function, the values in `right`'s members are used to initialize `student2`. Notice the `operator=` function has access to the `right` parameter's private members. Because the `operator=` function is a member of the `StudentTestScores` class, it has access to the private members of any `StudentTestScores` object that is passed into it.

Checking for Self-Assignment

Recall that `this` is a hidden argument that is passed to all nonstatic member functions. `this` is a special pointer that points to the object that called the function. Inside our `operator=` function, `this` will point to the object that is on the left side of the = sign, and `right` will reference the object on the right side of the = sign. This is illustrated in Figure 14-6. In the figure, assume that `student1` and `student2` are both `StudentTestScores` objects.

Figure 14-6 The `this` pointer and the `right` parameter



Look back at the `operator=` function, and notice the first statement inside the body of the function (line 64) is an `if` statement that starts like this:

```
if (this != &right)
```

This `if` statement is making sure the address stored in `this` is not the same as the address of the `right` object. In other words, it is checking for self-assignment. Self-assignment occurs when you assign an object to itself. For example, assume `student1` is a `StudentTestScores` object, and it holds a set of test scores. The following statement performs self-assignment:

```
student1 = student1;
```

Obviously, there is no reason to write a statement like this, but, in programs that work with pointers and references to objects, such assignments can happen in a roundabout way. It is important that you write code in your `operator=` functions to check for self-assignment, especially in classes that dynamically allocate memory. For example, the following code fragment shows how the `operator=` member function of the `StudentTestScores` class would look if we did not check for self-assignment:

```
// Overloaded = operator that does not check for self-assignment
const StudentTestScores operator=(const StudentTestScores &right)
{  delete[] testScores;
   studentName = right.studentName;
   numTestScores = right.numTestScores;
   testScores = new double[numTestScores];
   for (int i = 0; i < numTestScores; i++)
      testScores[i] = right.testScores[i];
   return *this;
}
```

In this version of the function, the first thing we do is delete the memory for the `testScores` array. In the event of self-assignment, this will be a problem because we are deleting the memory from which we will be copying! Obviously, the assignment will not work as intended. The corrected version of the function, shown again here, avoids the problem by skipping all of the assignment steps when self-assignment has been detected:

```
62  // Overloaded = operator
63  const StudentTestScores operator=(const StudentTestScores &right)
64  {  if (this != &right)
65     {
66        delete[] testScores;
67        studentName = right.studentName;
68        numTestScores = right.numTestScores;
69        testScores = new double[numTestScores];
70        for (int i = 0; i < numTestScores; i++)
71           testScores[i] = right.testScores[i];
72     }
73     return *this;
74  }
```

The = Operator's Return Value

The last thing we need to discuss about the `operator=` function is its return value. Recall that C++'s built-in = operator allows multiple assignment statements such as:

```
a = b = c = d;
```

This statement works in the following way:

- The expression `c = d` assigns `d` to `c`, then returns the new value of `c`.
- The new value of `c`, which is returned from the previous expression is assigned to `b`, then the new value of `b` is returned.
- The new value of `b`, which is returned from the previous expression, is assigned to `a`.

If a class object's overloaded `=` operator is to function this way, it too must return the value of the object that received the assignment. That's why the `operator=` function in the `StudentTestScores` class has a return type of `const StudentTestScores`, and the last statement in the function is:

```
return *this;
```

The expression `*this` dereferences the `this` pointer, giving us the actual object that received the assignment. A `const` copy of that object is then returned.

Copy Assignment

The overloaded `=` operator that we just looked at is an example of a *copy assignment operator*. It is called a copy assignment operator because it copies data from one object to another. Later in this chapter, you will see an example of another type of assignment known as *move assignment*.

Program 14-6 demonstrates the `StudentTestScores` class with its overloaded assignment operator. (This file can be found in the Student Source Code Folder Chapter 14\StudentTestScores Version 3.)

Program 14-6

```
1  // This program demonstrates the overloaded = operator
2  #include <iostream>
3  #include "StudentTestScores.h"
4  using namespace std;
5
6  // Function prototype
7  void displayStudent(StudentTestScores);
8
9  int main()
10 {
11     // Create a StudentTestScores object and
12     // assign test scores.
13     StudentTestScores student1("Kelly Thorton", 3);
14     student1.setTestScore(100.0, 0);
15     student1.setTestScore(95.0, 1);
16     student1.setTestScore(80, 2);
17
18     // Create another StudentTestScore object
19     // with default test scores.
20     StudentTestScores student2("Jimmy Griffin", 5);
21
22     // Assign the student1 object to student2
23     student2 = student1;
24
```

(program continues)

Program 14-6 *(continued)*

```
25      // Display both objects. They should
26      // contain the same data.
27      displayStudent(student1);
28      displayStudent(student2);
29      return 0;
30  }
31
32  // The displayStudent function accepts a
33  // StudentTestScores object's data.
34  void displayStudent(StudentTestScores s)
35  {
36      cout << "Name: " << s.getStudentName() << endl;
37      cout << "Test Scores: ";
38      for (int i = 0; i < s.getNumTestScores(); i++)
39          cout << s.getTestScore(i) << " ";
40      cout << endl;
41  }
```

Program Output

```
Name: Kelly Thorton
Test Scores: 100 95 80
Name: Kelly Thorton
Test Scores: 100 95 80
```

The overloaded = operator function is demonstrated again in Program 14-7. The multiple assignment statement in line 22 causes the `operator=` function to execute. (This file and the revised version of the `StudentTestScores` class can be found in the Student Source Code Folder Chapter 14\StudentTestScores Version 4.)

Program 14-7

```
1  // This program demonstrates the overloaded = operator returning a value.
2  #include <iostream>
3  #include "StudentTestScores.h"
4  using namespace std;
5
6  // Function prototype
7  void displayStudent(StudentTestScores);
8
9  int main()
10 {
11     // Create a StudentTestScores object.
12     StudentTestScores student1("Kelly Thorton", 3);
13     student1.setTestScore(100.0, 0);
14     student1.setTestScore(95.0, 1);
15     student1.setTestScore(80, 2);
16
17     // Create two more StudentTestScores objects.
18     StudentTestScores student2("Jimmy Griffin", 5);
19     StudentTestScores student3("Kristen Lee", 10);
20
```

```
21      // Assign student1 to student2 and student3.
22      student3 = student2 = student1;
23
24      // Display the objects.
25      displayStudent(student1);
26      displayStudent(student2);
27      displayStudent(student3);
28      return 0;
29  }
30
31  // displayStudent function
32  void displayStudent(StudentTestScores s)
33  {
34      cout << "Name: " << s.getStudentName() << endl;
35      cout << "Test Scores: ";
36      for (int i = 0; i < s.getNumTestScores(); i++)
37          cout << s.getTestScore(i) << " ";
38      cout << endl;
39  }
```

Program Output

```
Name: Kelly Thorton
Test Scores: 100 95 80
Name: Kelly Thorton
Test Scores: 100 95 80
Name: Kelly Thorton
Test Scores: 100 95 80
```

Some General Issues of Operator Overloading

Now that you have had a taste of operator overloading, let's look at some of the general issues involved in this programming technique.

Although it is not a good programming practice, you can change an operator's entire meaning if that's what you wish to do. There is nothing to prevent you from changing the = symbol from an assignment operator to a "display" operator. For instance, the following class does just that:

```
class Weird
{
private:
   int value;
public:
   Weird(int v)
      { value = v; }
   void operator=(const weird &right) const
      { cout << right.value << endl; }
};
```

Although the `operator=` function in the `Weird` class overloads the assignment operator, the function doesn't perform an assignment. Instead, it displays the contents of `right.value`. Consider the following program segment:

```
Weird a(5), b(10);
a = b;
```

Although the statement `a = b` looks like an assignment statement, it actually causes the contents of `b`'s `value` member to be displayed on the screen:

```
10
```

Another operator overloading issue is you cannot change the number of operands taken by an operator. The = symbol must always be a binary operator. Likewise, ++ and -- must always be unary operators.

The last issue is although you may overload most of the C++ operators, you cannot overload all of them. Table 14-1 shows all of the C++ operators that may be overloaded (some of the operators shown in the table are beyond the scope of this book and are not discussed).

Table 14-1 Operators that May Be Overloaded

+	-	*	/	%	^	&	|	~	!	=	<
>	+=	-=	*=	/=	%=	^=	&=	|=	<<	>>	>>=
<<=	==	!=	<=	>=	&&	||	++	--	-<*	,	-<
[]	()	new	delete								

The only operators that cannot be overloaded are

```
?:    .    .*    ::    sizeof
```

Overloading Math Operators

Many classes would benefit not only from an overloaded assignment operator, but also from overloaded math operators. To illustrate this, consider the `FeetInches` class shown in the following two files. (These files can be found in the Student Source Code Folder Chapter 14\FeetInches Version 1.)

Contents of `FeetInches.h` (Version 1)

```
 1  #ifndef FEETINCHES_H
 2  #define FEETINCHES_H
 3
 4  // The FeetInches class holds distances or measurements
 5  // expressed in feet and inches.
 6
 7  class FeetInches
 8  {
 9  private:
10      int feet;         // To hold a number of feet
11      int inches;       // To hold a number of inches
12      void simplify();  // Defined in FeetInches.cpp
13  public:
14      // Constructor
15      FeetInches(int f = 0, int i = 0)
16          { feet = f;
17            inches = i;
18            simplify(); }
19
```

```
20      // Mutator functions
21      void setFeet(int f)
22         { feet = f; }
23
24      void setInches(int i)
25         { inches = i;
26           simplify(); }
27
28      // Accessor functions
29      int getFeet() const
30         { return feet; }
31
32      int getInches() const
33         { return inches; }
34
35      // Overloaded operator functions
36      FeetInches operator + (const FeetInches &); // Overloaded +
37      FeetInches operator - (const FeetInches &); // Overloaded -
38  };
39
40  #endif
```

Contents of FeetInches.cpp (Version 1)

```
 1  // Implementation file for the FeetInches class
 2  #include <cstdlib>  // Needed for abs()
 3  #include "FeetInches.h"
 4
 5  //*************************************************************
 6  // Definition of member function simplify. This function      *
 7  // checks for values in the inches member greater than        *
 8  // twelve or less than zero. If such a value is found,        *
 9  // the numbers in feet and inches are adjusted to conform     *
10  // to a standard feet & inches expression. For example,       *
11  // 3 feet 14 inches would be adjusted to 4 feet 2 inches and  *
12  // 5 feet -2 inches would be adjusted to 4 feet 10 inches.    *
13  //*************************************************************
14
15  void FeetInches::simplify()
16  {
17      if (inches >= 12)
18      {
19          feet += (inches / 12);
20          inches = inches % 12;
21      }
22      else if (inches < 0)
23      {
24          feet -= ((abs(inches) / 12) + 1);
25          inches = 12 - (abs(inches) % 12);
26      }
27  }
28
```

```
29  //*********************************************
30  // Overloaded binary + operator.              *
31  //*********************************************
32
33  FeetInches FeetInches::operator + (const FeetInches &right)
34  {
35      FeetInches temp;
36
37      temp.inches = inches + right.inches;
38      temp.feet = feet + right.feet;
39      temp.simplify();
40      return temp;
41  }
42
43  //*********************************************
44  // Overloaded binary - operator.              *
45  //*********************************************
46
47  FeetInches FeetInches::operator - (const FeetInches &right)
48  {
49      FeetInches temp;
50
51      temp.inches = inches - right.inches;
52      temp.feet = feet - right.feet;
53      temp.simplify();
54      return temp;
55  }
```

The `FeetInches` class is designed to hold distances or measurements expressed in feet and inches. It consists of eight member functions:

- A constructor that allows the `feet` and `inches` members to be set. The default values for these members is zero.
- **`setFeet`**—a function for storing a value in the `feet` member
- **`setInches`**—a function for storing a value in the `inches` member
- **`getFeet`**—a function for returning the value in the `feet` member
- **`getInches`**—a function for returning the value in the `inches` member
- **`simplify`**—a function for normalizing the values held in `feet` and `inches`. This function adjusts any set of values where the `inches` member is greater than 12 or less than 0.
- **`operator +`**—a function that overloads the standard `+` math operator
- **`operator -`**—a function that overloads the standard `-` math operator



NOTE: The `simplify` function uses the standard library function `abs()` to get the absolute value of the `inches` member. The `abs()` function requires that `cstdlib` be included.

The overloaded `+` and `-` operators allow one `FeetInches` object to be added to or subtracted from another. For example, assume the `length1` and `length2` objects are defined and initialized as follows:

```
FeetInches length1(3, 5), length2(6, 3);
```

The length1 object is holding the value 3 feet 5 inches, and the length2 object is holding the value 6 feet 3 inches. Because the + operator is overloaded, we can add these two objects in a statement such as:

```
length3 = length1 + length2;
```

This statement will add the values of the length1 and length2 objects and store the result in the length3 object. After the statement executes, the length3 object will be set to 9 feet 8 inches.

The member function that overloads the + operator appears in lines 33 through 41 of the FeetInches.cpp file.

This function is called anytime the + operator is used with two FeetInches objects. Just like the overloaded = operator we defined in the previous section, this function has one parameter: a constant reference object named right. This parameter references the object on the right side of the operator. For example, when the following statement is executed, right will reference the length2 object:

```
length3 = length1 + length2;
```

As before, it might be helpful to think of the statement above as the following function call:

```
length3 = length1.operator+(length2);
```

The length2 object is being passed to the function's parameter, right. When the function finishes, it will return a FeetInches object to length3. Now let's see what is happening inside the function. First, notice a FeetInches object named temp is defined locally in line 35:

```
FeetInches temp;
```

This object is a temporary location for holding the results of the addition. Next, line 37 adds inches to right.inches and stores the result in temp.inches:

```
temp.inches = inches + right.inches;
```

The inches variable is a member of length1, the object making the function call. It is the object on the left side of the operator. right.inches references the inches member of length2. The next statement, in line 38, is very similar. It adds feet to right.feet and stores the result in temp.feet.

```
temp.feet = feet + right.feet;
```

At this point in the function, temp contains the sum of the feet and inches members of both objects in the expression. The next step is to adjust the values, so they conform to a normal value expressed in feet and inches. This is accomplished in line 39 by calling temp.simplify():

```
temp.simplify();
```

The last step, in line 40, is to return the value stored in temp:

```
return temp;
```

In the statement length3 = length1 + length2, the return statement in the operator function causes the values stored in temp to be returned to the length3 object.

Program 14-8 demonstrates the overloaded operators. (This file can be found in the Student Source Code Folder Chapter 14\FeetInches Version 1.)

Program 14-8

```
// This program demonstrates the FeetInches class's overloaded
// + and - operators.
#include <iostream>
#include "FeetInches.h"
using namespace std;

int main()
{
    int feet, inches; // To hold input for feet and inches

    // Create three FeetInches objects. The default arguments
    // for the constructor will be used.
    FeetInches first, second, third;

    // Get a distance from the user.
    cout << "Enter a distance in feet and inches: ";
    cin >> feet >> inches;

    // Store the distance in the first object.
    first.setFeet(feet);
    first.setInches(inches);

    // Get another distance from the user.
    cout << "Enter another distance in feet and inches: ";
    cin >> feet >> inches;

    // Store the distance in second.
    second.setFeet(feet);
    second.setInches(inches);

    // Assign first + second to third.
    third = first + second;

    // Display the result.
    cout << "first + second = ";
    cout << third.getFeet() << " feet, ";
    cout << third.getInches() << " inches.\n";

    // Assign first - second to third.
    third = first - second;

    // Display the result.
    cout << "first - second = ";
    cout << third.getFeet() << " feet, ";
    cout << third.getInches() << " inches.\n";

    return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter a distance in feet and inches: 6 5 Enter
Enter another distance in feet and inches: 3 10 Enter
first + second = 10 feet, 3 inches.
first - second = 2 feet, 7 inches.
```

Overloading the Prefix ++ Operator

Unary operators, such as ++ and – –, are overloaded in a fashion similar to the way binary operators are implemented. Because unary operators only affect the object making the operator function call, however, there is no need for a parameter. For example, let's say you wish to have a prefix increment operator for the `FeetInches` class. Assume the `FeetInches` object `distance` is set to the values 7 feet and 5 inches. A ++ operator function could be designed to increment the object's `inches` member. The following statement would cause `distance` to have the value 7 feet 6 inches:

```
++distance;
```

The following function overloads the prefix ++ operator to work in this fashion:

```
FeetInches FeetInches::operator++()
{
   ++inches;
   simplify();
   return *this;
}
```

This function first increments the object's `inches` member. The `simplify()` function is called then the dereferenced `this` pointer is returned. This allows the operator to perform properly in statements like this:

```
distance2 = ++distance1;
```

Remember, the statement above is equivalent to

```
distance2 = distance1.operator++();
```

Overloading the Postfix ++ Operator

Overloading the postfix ++ operator is only slightly different than overloading the prefix version. Here is the function that overloads the postfix operator with the `FeetInches` class:

```
FeetInches FeetInches::operator++(int)
{
   FeetInches temp(feet, inches);
   inches++;
   simplify();
   return temp;
}
```

The first difference you will notice is the use of a *dummy parameter*. The word `int` in the function's parentheses establishes a nameless integer parameter. When C++ sees this parameter in an operator function, it knows the function is designed to be used in postfix mode. The second difference is the use of a temporary local variable, the `temp` object. `temp` is initialized with the `feet` and `inches` values of the object making the function call. `temp`,

therefore, is a copy of the object being incremented, but before the increment takes place. After `inches` is incremented and the `simplify` function is called, the contents of `temp` is returned. This causes the postfix operator to behave correctly in a statement like this:

```
distance2 = distance1++;
```

You will find a version of the `FeetInches` class with the overloaded prefix and postfix ++ operators in the Student Source Code Folder Chapter 14\FeetInches Version 2. In that folder, you will also find Program 14-9, which demonstrates these overloaded operators.

Program 14-9

```
// This program demonstrates the FeetInches class's overloaded
// prefix and postfix ++ operators.
#include <iostream>
#include "FeetInches.h"
using namespace std;

int main()
{
    int count;  // Loop counter

    // Define a FeetInches object with the default
    // value of 0 feet, 0 inches.
    FeetInches first;

    // Define a FeetInches object with 1 foot 5 inches.
    FeetInches second(1, 5);

    // Use the prefix ++ operator.
    cout << "Demonstrating prefix ++ operator.\n";
    for (count = 0; count < 12; count++)
    {
        first = ++second;
        cout << "first: " << first.getFeet() << " feet, ";
        cout << first.getInches() << " inches. ";
        cout << "second: " << second.getFeet() << " feet, ";
        cout << second.getInches() << " inches.\n";
    }

    // Use the postfix ++ operator.
    cout << "\nDemonstrating postfix ++ operator.\n";
    for (count = 0; count < 12; count++)
    {
        first = second++;
        cout << "first: " << first.getFeet() << " feet, ";
        cout << first.getInches() << " inches. ";
        cout << "second: " << second.getFeet() << " feet, ";
        cout << second.getInches() << " inches.\n";
    }

    return 0;
}
```

Program Output

```
Demonstrating prefix ++ operator.
first: 1 feet 6 inches. second: 1 feet 6 inches.
first: 1 feet 7 inches. second: 1 feet 7 inches.
first: 1 feet 8 inches. second: 1 feet 8 inches.
first: 1 feet 9 inches. second: 1 feet 9 inches.
first: 1 feet 10 inches. second: 1 feet 10 inches.
first: 1 feet 11 inches. second: 1 feet 11 inches.
first: 2 feet 0 inches. second: 2 feet 0 inches.
first: 2 feet 1 inches. second: 2 feet 1 inches.
first: 2 feet 2 inches. second: 2 feet 2 inches.
first: 2 feet 3 inches. second: 2 feet 3 inches.
first: 2 feet 4 inches. second: 2 feet 4 inches.
first: 2 feet 5 inches. second: 2 feet 5 inches.

Demonstrating postfix ++ operator.
first: 2 feet 5 inches. second: 2 feet 6 inches.
first: 2 feet 6 inches. second: 2 feet 7 inches.
first: 2 feet 7 inches. second: 2 feet 8 inches.
first: 2 feet 8 inches. second: 2 feet 9 inches.
first: 2 feet 9 inches. second: 2 feet 10 inches.
first: 2 feet 10 inches. second: 2 feet 11 inches.
first: 2 feet 11 inches. second: 3 feet 0 inches.
first: 3 feet 0 inches. second: 3 feet 1 inches.
first: 3 feet 1 inches. second: 3 feet 2 inches.
first: 3 feet 2 inches. second: 3 feet 3 inches.
first: 3 feet 3 inches. second: 3 feet 4 inches.
first: 3 feet 4 inches. second: 3 feet 5 inches.
```



Checkpoint

14.14 Assume there is a class named `Pet`. Write the prototype for a member function of `Pet` that overloads the `=` operator.

14.15 Assume `dog` and `cat` are instances of the `Pet` class, which has overloaded the `=` operator. Rewrite the following statement so it appears in function call notation instead of operator notation:

```
dog = cat;
```

14.16 What is the disadvantage of an overloaded `=` operator returning `void`?

14.17 Describe the purpose of the `this` pointer.

14.18 The `this` pointer is automatically passed to what type of functions?

14.19 Assume there is a class named `Animal` that overloads the `=` and `+` operators. In the following statement, assume `cat`, `tiger`, and `wildcat` are all instances of the `Animal` class:

```
wildcat = cat + tiger;
```

Of the three objects, `wildcat`, `cat`, or `tiger`, which is calling the `operator+` function? Which object is passed as an argument into the function?

14.20 What does the use of a dummy parameter in a unary operator function indicate to the compiler?

Overloading Relational Operators

In addition to the assignment and math operators, relational operators may be overloaded. This capability allows classes to be compared in statements that use relational expressions such as:

```
if (distance1 < distance2)
{
   ... code ...
}
```

Overloaded relational operators are implemented like other binary operators. The only difference is that a relational operator function should always return a `true` or `false` value. The third version of the `FeetInches` class in the Student Source Code Folder Chapter 14\FeetInches Version 3 contains functions to overload the >, <, and == relational operators. Here is the function for overloading the > operator:

```
bool FeetInches::operator > (const FeetInches &right)
{
   bool status;

   if (feet > right.feet)
      status = true;
   else if (feet == right.feet &&  inches > right.inches)
      status = true;
   else
      status = false;

   return status;
}
```

As you can see, the function compares the `feet` member (and if necessary, the `inches` member) with that of the parameter. If the calling object contains a value greater than that of the parameter, `true` is returned. Otherwise, `false` is returned.

Here is the code that overloads the < operator:

```
bool FeetInches::operator < (const FeetInches &right)
{
   bool status;

   if (feet < right.feet)
      status = true;
   else if (feet == right.feet &&  inches < right.inches)
      status = true;
   else
      status = false;

   return status;
}
```

Here is the code that overloads the == operator:

```
bool FeetInches::operator == (const FeetInches &right)
{
   bool status;

   if (feet == right.feet &&  inches == right.inches)
      status = true;
   else
      status = false;
```

```
    return status;
}
```

Program 14-10 demonstrates these overloaded operators. (This file can also be found in the Student Source Code Folder Chapter 14\FeetInches Version 3.)

Program 14-10

```
// This program demonstrates the FeetInches class's overloaded
// relational operators.
#include <iostream>
#include "FeetInches.h"
using namespace std;

int main()
{
    int feet, inches; // To hold input for feet and inches

    // Create two FeetInches objects. The default arguments
    // for the constructor will be used.
    FeetInches first, second;

    // Get a distance from the user.
    cout << "Enter a distance in feet and inches: ";
    cin >> feet >> inches;

    // Store the distance in first.
    first.setFeet(feet);
    first.setInches(inches);

    // Get another distance.
    cout << "Enter another distance in feet and inches: ";
    cin >> feet >> inches;

    // Store the distance in second.
    second.setFeet(feet);
    second.setInches(inches);

    // Compare the two objects.
    if (first == second)
        cout << "first is equal to second.\n";
    if (first > second)
        cout << "first is greater than second.\n";
    if (first < second)
        cout << "first is less than second.\n";

    return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter a distance in feet and inches: 6 5 Enter
Enter another distance in feet and inches: 3 10 Enter
first is greater than second.
```

(program output continues)

Program 14-10 *(continued)*

Program Output with Different Example Input Shown in Bold

```
Enter a distance in feet and inches: 5 5 Enter
Enter another distance in feet and inches: 5 5 Enter
first is equal to second.
```

Program Output with Different Example Input Shown in Bold

```
Enter a distance in feet and inches: 3 4 Enter
Enter another distance in feet and inches: 3 7 Enter
first is less than second.
```

Overloading the << and >> Operators

Overloading the math and relational operators gives you the ability to write those types of expressions with class objects just as naturally as with integers, floats, and other built-in data types. If an object's primary data members are private, however, you still have to make explicit member function calls to send their values to `cout`. For example, assume `distance` is a `FeetInches` object. The following statements display its internal values:

```
cout << distance.getFeet() << " feet, ";
cout << distance.getInches() << "inches";
```

It is also necessary to explicitly call member functions to set a `FeetInches` object's data. For instance, the following statements set the `distance` object to user-specified values:

```
cout << "Enter a value in feet: ";
cin >> f;
distance.setFeet(f);
cout << "Enter a value in inches: ";
cin >> i;
distance.setInches(i);
```

By overloading the stream insertion operator (`<<`), you could send the `distance` object to `cout`, as shown in the following code, and have the screen output automatically formatted in the correct way:

```
cout << distance;
```

Likewise, by overloading the stream extraction operator (`>>`), the `distance` object could take values directly from `cin`, as shown here:

```
cin >> distance;
```

Overloading these operators is done in a slightly different way, however, than overloading other operators. These operators are actually part of the `ostream` and `istream` classes defined in the C++ runtime library. (The `cout` and `cin` objects are instances of `ostream` and `istream`.) You must write operator functions to overload the `ostream` version of `<<` and the `istream` version of `>>`, so they work directly with a class such as `FeetInches`. The `FeetInches` class in the Student Source Code Folder Chapter 14\FeetInches Version 4 contains functions to overload the `<<` and `>>` operators. Here is the function that overloads the `<<` operator:

```
ostream &operator << (ostream &strm, const FeetInches &obj)
{
   strm << obj.feet << " feet, " << obj.inches << " inches";
   return strm;
}
```

Notice the function has two parameters: an `ostream` reference object and a `const FeetInches` reference object. The `ostream` parameter will be a reference to the actual `ostream` object on the left side of the `<<` operator. The second parameter is a reference to a `FeetInches` object. This parameter will reference the object on the right side of the `<<` operator. This function tells C++ how to handle any expression that has the following form:

ostreamObject `<<` *FeetInchesObject*

So, when C++ encounters the following statement, it will call the overloaded `operator<<` function:

```
cout << distance;
```

Notice the function's return type is `ostream &`. This means the function returns a reference to an `ostream` object. When the `return strm;` statement executes, it doesn't return a copy of `strm`, but a reference to it. This allows you to chain together several expressions using the overloaded `<<` operator, such as:

```
cout << distance1 << " " << distance2 << endl;
```

Here is the function that overloads the stream extraction operator to work with the `FeetInches` class:

```
istream &operator >> (istream &strm, FeetInches &obj)
{
   // Prompt the user for the feet.
   cout << "Feet: ";
   strm >> obj.feet;

   // Prompt the user for the inches.
   cout << "Inches: ";
   strm >> obj.inches;

   // Normalize the values.
   obj.simplify();

   return strm;
}
```

The same principles hold true for this operator. It tells C++ how to handle any expression in the following form:

istreamObject `>>` *FeetInchesObject*

Once again, the function returns a reference to an `istream` object, so several of these expressions may be chained together.

You have probably realized that neither of these functions is quite ready to work, though. Both functions attempt to directly access the `FeetInches` object's private members. Because the functions aren't themselves members of the `FeetInches` class, they don't have this type of access. The next step is to make the operator functions friends of `FeetInches`. This is

shown in the following listing of the FeetInches class declaration. (This file can be found in the Student Source Code Folder Chapter 14\FeetInches Version 4.)



NOTE: Some compilers require you to prototype the >> and << operator functions outside the class. For this reason, we have added the following statements to the FeetInches.h class specification file.

```
class FeetInches;                          // Forward Declaration

// Function Prototypes for Overloaded Stream Operators
ostream &operator << (ostream &, const FeetInches &);
istream &operator >> (istream &, FeetInches &);
```

Contents of FeetInches.h (Version 4)

```
 1  #ifndef FEETINCHES_H
 2  #define FEETINCHES_H
 3
 4  #include <iostream>
 5  using namespace std;
 6
 7  class FeetInches; // Forward Declaration
 8
 9  // Function Prototypes for Overloaded Stream Operators
10  ostream &operator << (ostream &, const FeetInches &);
11  istream &operator >> (istream &, FeetInches &);
12
13  // The FeetInches class holds distances or measurements
14  // expressed in feet and inches.
15
16  class FeetInches
17  {
18  private:
19      int feet;          // To hold a number of feet
20      int inches;        // To hold a number of inches
21      void simplify();   // Defined in FeetInches.cpp
22  public:
23      // Constructor
24      FeetInches(int f = 0, int i = 0)
25          { feet = f;
26            inches = i;
27            simplify(); }
28
29      // Mutator functions
30      void setFeet(int f)
31          { feet = f; }
32
33      void setInches(int i)
34          { inches = i;
35            simplify(); }
36
37      // Accessor functions
```

```
38      int getFeet() const
39         { return feet; }
40
41      int getInches() const
42          { return inches; }
43
44      // Overloaded operator functions
45      FeetInches operator + (const FeetInches &); // Overloaded +
46      FeetInches operator - (const FeetInches &); // Overloaded -
47      FeetInches operator ++ ();                  // Prefix ++
48      FeetInches operator ++ (int);               // Postfix ++
49      bool operator > (const FeetInches &);    // Overloaded >
50      bool operator < (const FeetInches &);    // Overloaded <
51      bool operator == (const FeetInches &);   // Overloaded ==
52
53      // Friends
54      friend ostream &operator << (ostream &, const FeetInches &);
55      friend istream &operator >> (istream &, FeetInches &);
56  };
57
58  #endif
```

Lines 54 and 55 in the class declaration tell C++ to make the overloaded << and >> operator functions friends of the `FeetInches` class:

```
friend ostream &operator<<(ostream &, const FeetInches &);
friend istream &operator>>(istream &, FeetInches &);
```

These statements give the operator functions direct access to the `FeetInches` class's private members. Program 14-11 demonstrates how the overloaded operators work. (This file can also be found in the Student Source Code Folder Chapter 14\FeetInches Version 4.)

Program 14-11

```
1  // This program demonstrates the << and >> operators,
2  // overloaded to work with the FeetInches class.
3  #include <iostream>
4  #include "FeetInches.h"
5  using namespace std;
6
7  int main()
8  {
9      FeetInches first, second; // Define two objects.
10
11     // Get a distance for the first object.
12     cout << "Enter a distance in feet and inches.\n";
13     cin >> first;
14
15     // Get a distance for the second object.
16     cout << "Enter another distance in feet and inches.\n";
17     cin >> second;
18
19     // Display the values in the objects.
20     cout << "The values you entered are:\n";
```

(program continues)

Program 14-11 *(continued)*

```
21       cout << first << " and " << second << endl;
22       return 0;
23   }
```

Program Output with Example Input Shown in Bold

```
Enter a distance in feet and inches.
Feet: 6 Enter
Inches: 5 Enter
Enter another distance in feet and inches.
Feet: 3 Enter
Inches: 10 Enter
The values you entered are:
6 feet, 5 inches and 3 feet, 10 inches
```

Overloading the [ ] Operator

In addition to the traditional operators, C++ allows you to change the way the [] symbols work. This gives you the ability to write classes that have array-like behaviors. For example, the `string` class overloads the [] operator, so you can access the individual characters stored in `string` class objects. Assume the following definition exists in a program:

```
string name = "William";
```

The first character in the string, 'W,' is stored at `name[0]`, so the following statement will display W on the screen.

```
cout << name[0];
```

You can use the overloaded [] operator to create an array class, like the following one. The class behaves like a regular array, but performs the bounds checking that C++ lacks.

Contents of `IntArray.h`

```
 1  // Specification file for the IntArray class
 2  #ifndef INTARRAY_H
 3  #define INTARRAY_H
 4
 5  class IntArray
 6  {
 7  private:
 8    int *aptr;                                  // Pointer to the array
 9    intarraySize;                               // Holds the array size
10    void subscriptError();                      // Handles invalid subscripts
11  public:
12    IntArray(int);                              // Constructor
13    IntArray(const IntArray &);                 // Copy constructor
14    ~IntArray();                                // Destructor
15
16    int size() const                            // Returns the array size
17      { return arraySize; }
18
19    const IntArray operator=(const IntArray &); // Overloaded = operator
20    int &operator[](const int &);               // Overloaded [] operator
21  };
22  #endif
```

Contents of IntArray.cpp

```
1  // Implementation file for the IntArray class
2  #include <iostream>
3  #include <cstdlib>   // For the exit function
4  #include "IntArray.h"
5  using namespace std;
6
7  //*********************************************************
8  // Constructor for IntArray class. Sets the size of the *
9  // array and allocates memory for it.                    *
10 //*********************************************************
11 IntArray::IntArray(int s)
12 {
13    arraySize = s;
14    aptr = new int[s];
15    for (int count = 0; count < arraySize; count++)
16       *(aptr + count) = 0;
17 }
18
19 //*********************************************************
20 // Copy Constructor for IntArray class.                  *
21 //*********************************************************
22 IntArray::IntArray(const IntArray &obj)
23 {
24    arraySize = obj.arraySize;
25    aptr = new int[arraySize];
26    for(int count = 0; count < arraySize; count++)
27      *(aptr + count) = *(obj.aptr + count);
28 }
29
30 //*********************************************************
31 // Destructor for IntArray class.                        *
32 //*********************************************************
33 IntArray::~IntArray()
34 {
35    if (arraySize > 0)
36       delete [] aptr;
37 }
38
39 //************************************************************
40 // subscriptError function. Displays an error message and    *
41 // terminates the program when a subscript is out of range. *
42 //************************************************************
43 void IntArray::subscriptError()
44 {
45    cout << "ERROR: Subscript out of range.\n";
46    exit(0);
47 }
48
49 //*********************************************************
50 // Overloaded = operator                                 *
51 //*********************************************************
52 const IntArray IntArray::operator=(const IntArray &right)
```

```
53  {
54     if (this != &right)
55     {
56        delete[] aptr;
57        arraySize = right.arraySize;
58        aptr = new int[arraySize];
59        for (int count = 0; count < arraySize; count++)
60           *(aptr + count) = *(right.aptr + count);
61     }
62     return *this;
63  }
64
65  //*********************************************************
66  // Overloaded [] operator. The argument is a subscript.  *
67  // This function returns a reference to the element      *
68  // in the array indexed by the subscript.                *
69  //*********************************************************
70  int &IntArray::operator[](const int &sub)
71  {
72     if (sub < 0 || sub >= arraySize)
73        subscriptError();
74     return aptr[sub];
75  }
```

Before focusing on the overloaded `[]` operator, let's look at the constructors and the destructor. The code for the first constructor in lines 11 through 17 of the IntArray.cpp file follows:

```
IntArray::IntArray(int s)
{
   arraySize = s;
   aptr = new int[s];
   for (int count = 0; count < arraySize; count++)
      *(aptr + count) = 0;
}
```

When an instance of the class is defined, the required number of elements is passed into the constructor's parameter, `s`. This value is assigned to the `arraySize` member, then used to dynamically allocate enough memory for the array. The constructor's final step is to store zeros in all of the array's elements.

The class also has a copy constructor in lines 22 through 28 which is used when a class object is initialized with another object's data:

```
IntArray::IntArray(const IntArray &obj)
{
   arraySize = obj.arraySize;
   aptr = new int[arraySize];
   for(int count = 0; count < arraySize; count++)
      *(aptr + count) = *(obj.aptr + count);
}
```

A reference to the initializing object is passed into the parameter `obj`. Once the memory is successfully allocated for the array, the constructor copies all the values in `obj`'s array into the calling object's array.

The destructor, in lines 33 through 37, simply frees the memory allocated by the class's constructors. First, however, it checks the value in `arraySize` to be sure the array has at least one element:

```
IntArray::~IntArray()
{
   if (arraySize > 0)
      delete [] aptr;
}
```

The `[]` operator is overloaded similarly to other operators. The definition of the `operator[]` function appears in lines 70 through 75:

```
int &IntArray::operator[](const int &sub)
{
   if (sub < 0 || sub >= arraySize)
      subscriptError();
   return aptr[sub];
}
```

The `operator[]` function can have only a single parameter. The one shown uses a constant reference to an integer. This parameter holds the value placed inside the brackets in an expression. For example, if `table` is an `IntArray` object, the number 12 will be passed into the `sub` parameter in the following statement:

```
cout << table[12];
```

Inside the function, the value in the `sub` parameter is tested by the following `if` statement:

```
if (sub < 0 || sub >= arraySize)
   subscriptError();
```

This statement determines whether `sub` is within the range of the array's subscripts. If `sub` is less than 0, or greater than or equal to `arraySize`, it's not a valid subscript, so the `subscriptError` function is called. If `sub` is within range, the function uses it as an offset into the array and returns a reference to the value stored at that location.

One critically important aspect of the function above is its return type. It's crucial that the function return not simply an integer, but a *reference* to an integer. The reason for this is that expressions such as the following must be possible:

```
table[5] = 27;
```

Remember, the built-in = operator requires the object on its left to be an lvalue. An lvalue must represent a modifiable memory location, such as a variable. The integer return value of a function is not an lvalue. If the `operator[]` function merely returns an integer, it cannot be used to create expressions placed on the left side of an assignment operator.

A reference to an integer, however, is an lvalue. If the `operator[]` function returns a reference, it can be used to create expressions like the following:

```
table[7] = 52;
```

In this statement, the `operator[]` function is called with 7 passed as its argument. Assuming 7 is within range, the function returns a reference to the integer stored at `(aptr + 7)`. In essence, the statement above is equivalent to:

```
*(aptr + 7) = 52;
```

Because the `operator[]` function returns actual integers stored in the array, it is not necessary for math or relational operators to be overloaded. Even the stream operators `<<` and `>>` will work just as they are with the `IntArray` class.

Program 14-12 demonstrates how the class works.

Program 14-12

```
// This program demonstrates an overloaded [] operator.
#include <iostream>
#include "IntArray.h"
using namespace std;

int main()
{
    const int SIZE = 10;  // Array size

    // Define an IntArray with 10 elements.
    IntArray table(SIZE);

    // Store values in the array.
    for (int x = 0; x < SIZE; x++)
        table[x] = (x * 2);

    // Display the values in the array.
    for (int x = 0; x < SIZE; x++)
        cout << table[x] << " ";
    cout << endl;

    // Use the standard + operator on array elements.
    for (int x = 0; x < SIZE; x++)
        table[x] = table[x] + 5;

    // Display the values in the array.
    for (int x = 0; x < SIZE; x++)
        cout << table[x] << " ";
    cout << endl;

    // Use the standard ++ operator on array elements.
    for (int x = 0; x < SIZE; x++)
        table[x]++;

    // Display the values in the array.
    for (int x = 0; x < SIZE; x++)
        cout << table[x] << " ";
    cout << endl;

    return 0;
}
```

Program Output

```
0 2 4 6 8 10 12 14 16 18
5 7 9 11 13 15 17 19 21 23
6 8 10 12 14 16 18 20 22 24
```

Program 14-13 demonstrates the `IntArray` class's bounds-checking capability.

Program 14-13

```
1  // This program demonstrates the IntArray class's bounds-checking ability.
2  #include <iostream>
3  #include "IntArray.h"
4  using namespace std;
5
6  int main()
7  {
8      const int SIZE = 10;  // Array size
9
10     // Define an IntArray with 10 elements.
11     IntArray table(SIZE);
12
13     // Store values in the array.
14     for (int x = 0; x < SIZE; x++)
15         table[x] = x;
16
17     // Display the values in the array.
18     for (int x = 0; x < SIZE; x++)
19         cout << table[x] << " ";
20     cout << endl;
21
22     // Attempt to use an invalid subscript.
23     cout << "Now attempting to use an invalid subscript.\n";
24     table[SIZE + 1] = 0;
25     return 0;
26 }
```

Program Output

```
0 1 2 3 4 5 6 7 8 9
Now attempting to use an invalid subscript.
ERROR: Subscript out of range.
```



Checkpoint

14.21 Describe the values that should be returned from functions that overload relational operators.

14.22 What is the advantage of overloading the << and >> operators?

14.23 What type of object should an overloaded << operator function return?

14.24 What type of object should an overloaded >> operator function return?

14.25 If an overloaded << or >> operator accesses a private member of a class, what must be done in that class's declaration?

14.26 Assume the class `NumList` has overloaded the `[]` operator. In the expression below, `list1` is an instance of the `NumList` class:

```
list1[25]
```

Rewrite the expression above to explicitly call the function that overloads the `[]` operator.

14.6 Object Conversion

CONCEPT: Special operator functions may be written to convert a class object to any other type.

As you've already seen, operator functions allow classes to work more like built-in data types. Another capability that operator functions can give classes is automatic type conversion.

Data type conversion happens "behind the scenes" with the built-in data types. For instance, suppose a program uses the following variables:

```
int i;
double d;
```

The statement below automatically converts the value in `i` to a floating-point number and stores it in `d`:

```
d = i;
```

Likewise, the following statement converts the value in `d` to an integer (truncating the fractional part) and stores it in `i`:

```
i = d;
```

The same functionality can also be given to class objects. For example, assuming `distance` is a `FeetInches` object and `d` is a `double`, the following statement would conveniently convert `distance`'s value into a floating-point number and store it in `d`, if `FeetInches` is properly written:

```
d = distance;
```

To be able to use a statement such as this, an operator function must be written to perform the conversion. The Student Source Code Folder Chapter 14\FeetInches Version 5 contains a version of the `FeetInches` class with such an operator function. Here is the code for the operator function that converts a `FeetInches` object to a `double`:

```
FeetInches::operator double()
{
   double temp = feet;

   temp += (inches / 12.0);
   return temp;
}
```

This function contains an algorithm that will calculate the decimal equivalent of a feet and inches measurement. For example, the value 4 feet 6 inches will be converted to 4.5. This value is stored in the local variable `temp`. The `temp` variable is then returned.



NOTE: No return type is specified in the function header. Because the function is a `FeetInches`-to-`double` conversion function, it will always return a `double`. Also, because the function takes no arguments, there are no parameters.

The revised FeetInches class also has an operator function that converts a FeetInches object to an int. The function, shown here, simply returns the feet member, thus truncating the inches value:

```
FeetInches:: operator int()
{
   return feet;
}
```

Program 14-14 demonstrates both of these conversion functions. (This file can also be found in the Student Source Code Folder Chapter 14\FeetInches Version 5.)

Program 14-14

```
 1  // This program demonstrates the FeetInches class's
 2  // conversion functions.
 3  #include <iostream>
 4  #include "FeetInches.h"
 5  using namespace std;
 6
 7  int main()
 8  {
 9      double d;  // To hold double input
10      int i;     // To hold int input
11
12      // Define a FeetInches object.
13      FeetInches distance;
14
15      // Get a distance from the user.
16      cout << "Enter a distance in feet and inches:\n";
17      cin >> distance;
18
19      // Convert the distance object to a double.
20      d = distance;
21
22      // Convert the distance object to an int.
23      i = distance;
24
25      // Display the values.
26      cout << "The value " << distance;
27      cout << " is equivalent to " << d << " feet\n";
28      cout << "or " << i << " feet, rounded down.\n";
29      return 0;
30  }
```

Program Output with Example Input Shown in Bold

```
Enter a distance in feet and inches:
Feet: 8 Enter
Inches: 6 Enter
The value 8 feet, 6 inches is equivalent to 8.5 feet
or 8 feet, rounded down.
```



Checkpoint

14.27 When overloading a binary operator such as + or –, what object is passed into the operator function's parameter?

14.28 Explain why overloaded prefix and postfix ++ and –– operator functions should return a value.

14.29 How does C++ tell the difference between an overloaded prefix and postfix ++ or –– operator function?

14.30 Write member functions of the `FeetInches` class that overload the prefix and postfix –– operators. Demonstrate the functions in a simple program similar to Program 14-14.

14.7 Aggregation

CONCEPT: Aggregation occurs when a class contains an instance of another class.



VideoNote
Class Aggregation

In real life, objects are frequently made of other objects. A house, for example, is made of door objects, window objects, wall objects, and much more. It is the combination of all these objects that makes a house object.

When designing software, it sometimes makes sense to create an object from other objects. For example, suppose you need an object to represent a course that you are taking in college. You decide to create a `Course` class, which will hold the following information:

- The course name
- The instructor's last name, first name, and office number
- The textbook's title, author, and publisher

In addition to the course name, the class will hold items related to the instructor and the textbook. You could put attributes for each of these items in the `Course` class. However, a good design principle is to separate related items into their own classes. In this example, an `Instructor` class could be created to hold the instructor-related data, and a `TextBook` class could be created to hold the textbook-related data. Instances of these classes could then be used as attributes in the `Course` class.

Let's take a closer look at how this might be done. To keep things simple, the `Instructor` class will have only the following functions:

- A default constructor that assigns empty strings to the instructor's last name, first name, and office number.
- A constructor that accepts arguments for the instructor's last name, first name, and office number
- **`set`**—a function that can be used to set all of the class's attributes
- **`print`**—a function that displays the object's attribute values

The code for the `Instructor` class is shown here:

Contents of `Instructor.h`

```
#ifndef INSTRUCTOR
#define INSTRUCTOR
#include <iostream>
#include <string>
using namespace std;

// Instructor class
class Instructor
{
private:
    string lastName;      // Last name
    string firstName;     // First name
    string officeNumber;  // Office number
public:
    // The default constructor stores empty strings
    // in the string objects.
    Instructor()
       { set("", "", ""); }

    // Constructor
    Instructor(string lname, string fname, string office)
       { set(lname, fname, office); }

    // set function
    void set(string lname, string fname, string office)
       { lastName = lname;
         firstName = fname;
         officeNumber = office; }

     // print function
     void print() const
        { cout << "Last name: " << lastName << endl;
          cout << "First name: " << firstName << endl;
          cout << "Office number: " << officeNumber << endl; }
};
#endif
```

The code for the `TextBook` class is shown next. As before, we want to keep the class simple. The only functions it has are a default constructor, a constructor that accepts arguments, a `set` function, and a `print` function.

Contents of `TextBook.h`

```
#ifndef TEXTBOOK
#define TEXTBOOK
#include <iostream>
#include <string>
using namespace std;

// TextBook class
class TextBook
```

```
 9  {
10  private:
11       string title;      // Book title
12       string author;     // Author name
13       string publisher;  // Publisher name
14  public:
15       // The default constructor stores empty strings
16       // in the string objects.
17       TextBook()
18          { set("", "", ""); }
19
20       // Constructor
21       TextBook(string textTitle, string auth, string pub)
22          { set(textTitle, auth, pub); }
23
24       // set function
25       void set(string textTitle, string auth, string pub)
26          { title = textTitle;
27            author = auth;
28            publisher = pub; }
29
30       // print function
31       void print() const
32          { cout << "Title: " << title << endl;
33            cout << "Author: " << author << endl;
34            cout << "Publisher: " << publisher << endl; }
35  };
36  #endif
```

The `Course` class is shown next. Notice the `Course` class has an `Instructor` object and a `TextBook` object as member variables. Those objects are used as attributes of the `Course` object. Making an instance of one class an attribute of another class is called *object aggregation*. The word *aggregate* means "a whole that is made of constituent parts." In this example, the `Course` class is an aggregate class because an instance of it is made of constituent objects.

When an instance of one class is a member of another class, it is said that there is a "has a" relationship between the classes. For example, the relationships that exist among the `Course`, `Instructor`, and `TextBook` classes can be described as follows:

- The course *has an* instructor.
- The course *has a* textbook.

The "has a" relationship is sometimes called a *whole–part relationship* because one object is part of a greater whole.

Contents of `Course.h`

```
1  #ifndef COURSE
2  #define COURSE
3  #include <iostream>
4  #include <string>
5  #include "Instructor.h"
6  #include "TextBook.h"
7  using namespace std;
```

```
 8
 9  class Course
10  {
11  private:
12      string courseName;       // Course name
13      Instructor instructor;   // Instructor
14      TextBook textbook;       // Textbook
15  public:
16      // Constructor
17      Course(string course, string instrLastName,
18             string instrFirstName, string instrOffice,
19             string textTitle, string author,
20             string publisher)
21      { // Assign the course name.
22        courseName = course;
23
24        // Assign the instructor.
25        instructor.set(instrLastName, instrFirstName, instrOffice);
26
27        // Assign the textbook.
28        textbook.set(textTitle, author, publisher); }
29
30      // print function
31      void print() const
32      { cout << "Course name: " << courseName << endl << endl;
33        cout << "Instructor Information:\n";
34        instructor.print();
35        cout << "\nTextbook Information:\n";
36        textbook.print();
37        cout << endl;}
38  };
39  #endif
```

Program 14-15 demonstrates the `Course` class.

Program 14-15

```
1  // This program demonstrates the Course class.
2  #include "Course.h"
3
4  int main()
5  {
6      // Create a Course object.
7      Course myCourse("Intro to Computer Science", // Course name
8          "Kramer", "Shawn", "RH3010",       // Instructor info
9          "Starting Out with C++", "Gaddis", // Textbook title and author
10         "Pearson");                        // Textbook publisher
11
12     // Display the course info.
13     myCourse.print();
14     return 0;
15 }
```

(program output continues)

Program 14-15 *(continued)*

Program Output

```
Course name: Intro to Computer Science

Instructor Information:
Last name: Kramer
First name: Shawn
Office number: RH3010

Textbook Information:
Title: Starting Out with C++
Author: Gaddis
Publisher: Pearson
```

Using Member Initialization Lists with Aggregate Classes

Recall from Chapter 13 that in a constructor, you can use a member initialization list as an alternative way to initialize the class's member variables. An aggregate class's constructor can also use a member initialization list to call the constructors for each of its member objects. The following shows how we could rewrite the constructor for the `Course` class, using a member initialization list:

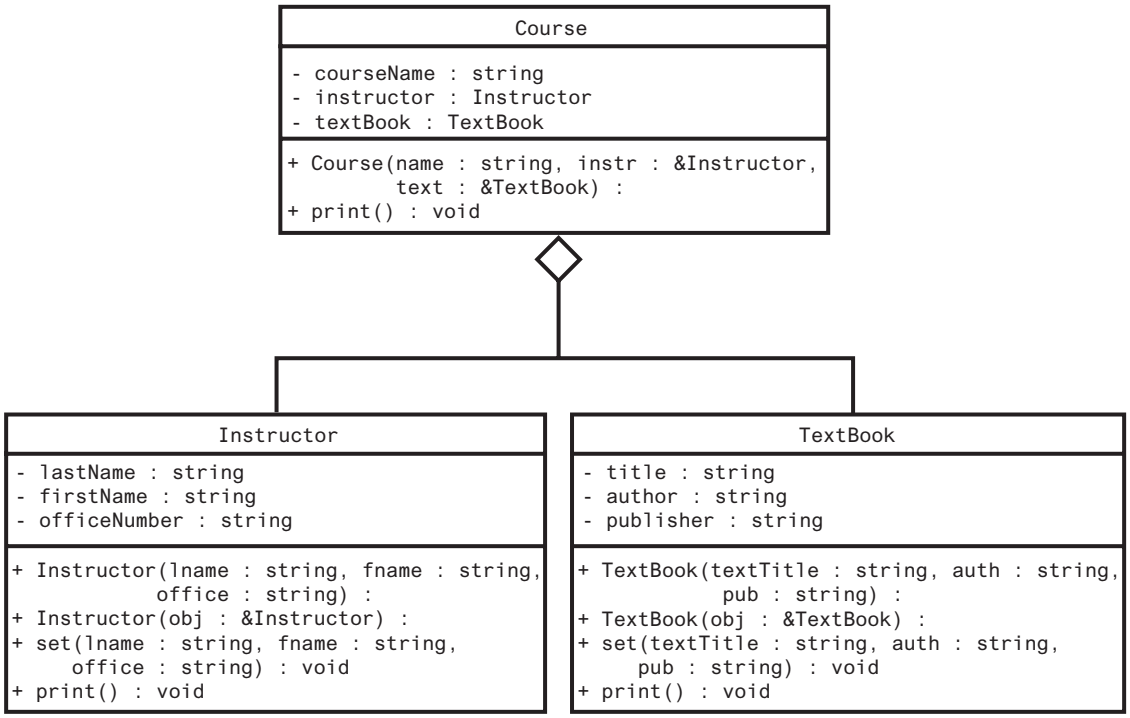
```
1  Course(string course, string instrLastName, string instrFirstName,
2         string instrOffice, string textTitle, string author,
3         string publisher) :
4  instructor(instrLastName, instrFirstName, instrOffice),
5  textbook(textTitle, author, publisher)
6  {
7      // Assign the course name.
8      courseName = course;
9  }
```

In this version of the constructor, a colon appears at the end of the function header (line 3), followed by the member initialization list (lines 4 and 5). Line 4 calls the `instructor` member's constructor, passing the necessary arguments, and line 5 calls the `textbook` member's constructor, passing the necessary arguments. Notice in the initialization list we are using the names of the member objects, rather than their classes. This allows constructors of different objects of the same class to be invoked in the same initialization list.

Aggregation in UML Diagrams

In Chapter 13, you were introduced to the Unified Modeling Language (UML) as a tool for designing classes. You show aggregation in a UML diagram by connecting two classes with a line that has an open diamond at one end. The diamond is closest to the class that is the aggregate. Figure 14-7 shows a UML diagram depicting the relationship between the `Course`, `Instructor`, and `TextBook` classes. The open diamond is closest to the `Course` class because it is the aggregate (the whole).

Figure 14-7 UML diagram showing aggregation



14.8 Focus on Object-Oriented Design: Class Collaborations

CONCEPT: **It is common for classes to interact, or collaborate, with one another to perform their operations. Part of the object-oriented design process is identifying the collaborations between classes.**

In an object-oriented application, it is common for objects of different classes to collaborate. This simply means objects interact with each other. Sometimes one object will need the services of another object in order to fulfill its responsibilities. For example, let's say an object needs to read a number from the keyboard then format the number to appear as a dollar amount. The object might use the services of the `cin` object to read the number from the keyboard then use the services of another object that is designed to format the number.

If one object is to collaborate with another object, then it must know something about the other object's member functions and how to call them. Let's look at an example.

The following code shows a class named `Stock`. An object of this class holds data about a company's stock. This class has two attributes: `symbol` and `sharePrice`. The `symbol` attribute holds the trading symbol for the company's stock. This is a short series of characters that are used to identify the stock on the stock exchange. For example, the XYZ Company's stock might have the trading symbol XYZ. The `sharePrice` attribute holds the current price per share of the stock. The class also has the following member functions:

- A default constructor that initializes `symbol` to an empty string and `sharePrice` to 0.0.
- A constructor that accepts arguments for the symbol and share price.
- A copy constructor.
- **`set`**—a function that accepts arguments for the symbol and share price
- **`getSymbol`**—a function that returns the stock's trading symbol
- **`getSharePrice`**—a function that returns the current price of the stock

Contents of `Stock.h`

```
#ifndef STOCK
#define STOCK
#include <string>
using namespace std;

class Stock
{
private:
     string symbol;         // Trading symbol of the stock
     double sharePrice;     // Current price per share
public:
     // Default constructor
     Stock()
          { set("", 0.0);}

     // Constructor
     Stock(const string sym, double price)
          { set(sym, price); }

     // Copy constructor
     Stock(const Stock &obj)
          { set(obj.symbol, obj.sharePrice); }

     // Mutator function
     void set(string sym, double price)
          { symbol = sym;
            sharePrice = price; }

     // Accessor functions
     string getSymbol() const
          { return symbol; }
```

```
32
33      double getSharePrice() const
34          { return sharePrice; }
35  };
36  #endif
```

The following code shows another class named `StockPurchase` that uses an object of the `Stock` class to simulate the purchase of a stock. The `StockPurchase` class is responsible for calculating the cost of the stock purchase. To do that, the `StockPurchase` class must know how to call the `Stock` class's `getSharePrice` function to get the price per share of the stock.

Contents of StockPurchase.h

```
1   #ifndef STOCK_PURCHASE
2   #define STOCK_PURCHASE
3   #include "Stock.h"
4
5   class StockPurchase
6   {
7   private:
8       Stock stock;   // The stock that was purchased
9       int shares;    // The number of shares
10  public:
11      // The default constructor sets shares to 0. The stock
12      // object is initialized by its default constructor.
13      StockPurchase()
14          { shares = 0;}
15
16      // Constructor
17      StockPurchase(const Stock &stockObject, int numShares)
18          { stock = stockObject;
19            shares = numShares; }
20
21      // Accessor function
22      double getCost() const
23          { return shares * stock.getSharePrice(); }
24  };
25  #endif
```

The second constructor for the `StockPurchase` class accepts a `Stock` object representing the stock that is being purchased, and an `int` representing the number of shares to purchase. In line 18, we see the first collaboration: the `StockPurchase` constructor makes a copy of the `Stock` object by using the `Stock` class's copy constructor. The next collaboration takes place in the `getCost` function. This function calculates and returns the cost of the stock purchase. In line 23, it calls the `Stock` class's `getSharePrice` function to determine the stock's price per share. Program 14-16 demonstrates this class.

Program 14-16

```
// Stock trader program
#include <iostream>
#include <iomanip>
#include "Stock.h"
#include "StockPurchase.h"
using namespace std;

int main()
{
    int sharesToBuy;  // Number of shares to buy

    // Create a Stock object for the company stock. The
    // trading symbol is XYZ and the stock is currently
    // priced at $9.62 per share.
    Stock xyzCompany("XYZ", 9.62);

    // Display the symbol and current share price.
    cout << setprecision(2) << fixed << showpoint;
    cout << "XYZ Company's trading symbol is "
         << xyzCompany.getSymbol() << endl;
    cout << "The stock is currently $"
         << xyzCompany.getSharePrice()
         << " per share.\n";

    // Get the number of shares to purchase.
    cout << "How many shares do you want to buy? ";
    cin >> sharesToBuy;

    // Create a StockPurchase object for the transaction.
    StockPurchase buy(xyzCompany, sharesToBuy);

    // Display the cost of the transaction.
    cout << "The cost of the transaction is $"
         << buy.getCost() << endl;
    return 0;
}
```

Program Output with Example Input Shown in Bold

```
XYZ Company's trading symbol is XYZ
The stock is currently $9.62 per share.
How many shares do you want to buy? 100 Enter
The cost of the transaction is $962.00
```

Determining Class Collaborations with CRC Cards

During the object-oriented design process, you can determine many of the collaborations that will be necessary between classes by examining the responsibilities of the classes. In Chapter 13, we discussed the process of finding the classes and their responsibilities. Recall from that section that a class's responsibilities are:

- the things that the class is responsible for knowing.
- the actions that the class is responsible for doing.

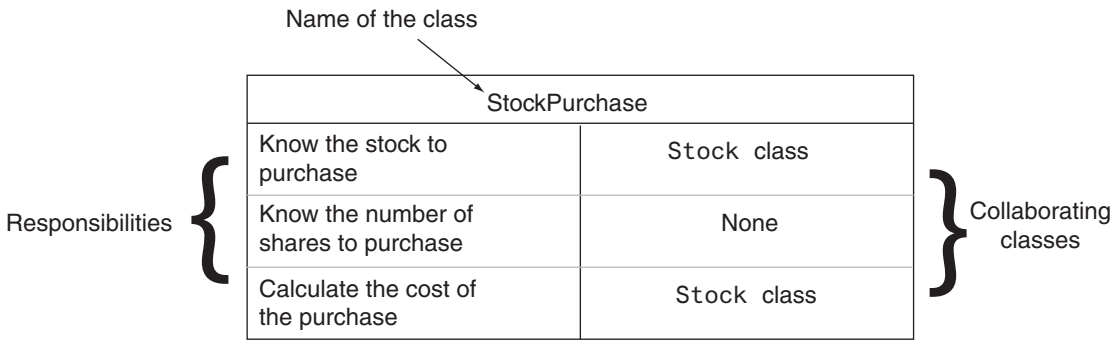
Often you will determine that the class must collaborate with another class in order to fulfill one or more of its responsibilities. One popular method of discovering a class's responsibilities and collaborations is by creating CRC cards. *CRC* stands for class, responsibilities, and collaborations.

You can use simple index cards for this procedure. Once you have gone through the process of finding the classes (which was discussed in Chapter 13), set aside one index card for each class. At the top of the index card, write the name of the class. Divide the rest of the card into two columns. In the left column, write each of the class's responsibilities. As you write each responsibility, think about whether the class needs to collaborate with another class to fulfill that responsibility. Ask yourself questions such as:

- Will an object of this class need to get data from another object in order to fulfill this responsibility?
- Will an object of this class need to request another object to perform an operation in order to fulfill this responsibility?

If collaboration is required, write the name of the collaborating class in the right column, next to the responsibility that requires it. If no collaboration is required for a responsibility, simply write "None" in the right column, or leave it blank. Figure 14-8 shows an example CRC card for the `StockPurchase` class.

Figure 14-8 CRC card



From the CRC card shown in the figure, we can see that the `StockPurchase` class has the following responsibilities and collaborations:

- Responsibility: To know the stock to purchase
  Collaboration: The `Stock` class
- Responsibility: To know the number of shares to purchase
  Collaboration: None
- Responsibility: To calculate the cost of the purchase
  Collaboration: The `Stock` class

When you have completed a CRC card for each class in the application, you will have a good idea of each class's responsibilities and how the classes must interact.



Checkpoint

14.31 What are the benefits of having operator functions that perform object conversion?

14.32 Why are no return types listed in the prototypes or headers of operator functions that perform data type conversion?

14.33 Assume there is a class named `BlackBox`. Write the header for a member function that converts a `BlackBox` object to an `int`.

14.34 Assume there are two classes, `Big` and `Small`. The `Big` class has, as a member, an instance of the `Small` class. Write a sentence that describes the relationship between the two classes.

14.9 Focus on Object-Oriented Programming: Simulating the Game of Cho-Han

Cho-Han is a traditional Japanese gambling game in which a dealer uses a cup to roll two six-sided dice. The cup is placed upside down on a table so the value of the dice is concealed. Players then wager on whether the sum of the dice values is even (Cho) or odd (Han). The winner, or winners, take all of the wagers, or the house takes them if there are no winners.

We will develop a program that simulates a simplified variation of the game. The simulated game will have a dealer and two players. The players will not wager money, but will simply guess whether the sum of the dice values is even (Cho) or odd (Han). One point will be awarded to the player, or players, for correctly guessing the outcome. The game is played for five rounds, and the player with the most points is the grand winner.

In the program, we will use the `Die` class introduced in Chapter 13 to simulate the dice. We will create two instances of the `Die` class to represent two six-sided dice. In addition to the `Die` class, we will write the following classes:

- `Dealer` class: We will create an instance of this class to represent the dealer. It will have the ability to roll the dice, report the value of the dice, and report whether the total dice value is Cho or Han.
- `Player` class: We will create two instances of this class to represent the players. Instances of the `Player` class can store the player's name, make a guess between Cho or Han, and be awarded points.

First, let's look at the `Dealer` class.

Contents of `Dealer.h`

```
1  // Specification file for the Dealer class
2  #ifndef DEALER_H
3  #define DEALER_H
4  #include <string>
5  #include "Die.h"
6  using namespace std;
7
```

```
 8  class Dealer
 9  {
10  private:
11      Die die1;              // Object for die #1
12      Die die2;              // Object for die #2
13      int die1Value;         // Value of die #1
14      int die2Value;         // Value of die #2
15
16  public:
17      Dealer();              // Constructor
18      void rollDice();       // To roll the dice
19      string getChoOrHan();  // To get the result (Cho or Han)
20      int getDie1Value();    // To get the value of die #1
21      int getDie2Value();    // To get the value of die #2
22  };
23  #endif
```

Contents of `Dealer.cpp`

```
 1  // Implementation file for the Dealer class
 2  #include "Dealer.h"
 3  #include <string>
 4  using namespace std;
 5
 6  //*****************************************
 7  // Constructor                            *
 8  //*****************************************
 9  Dealer::Dealer()
10  {
11      // Set the intial dice values to 0.
12      // (We will not use these values.)
13      die1Value = 0;
14      die2Value = 0;
15  }
16
17  //*****************************************
18  // The rollDice member function rolls the  *
19  // dice and saves their values.            *
20  //*****************************************
21  void Dealer::rollDice()
22  {
23      // Roll the dice.
24      die1.roll();
25      die2.roll();
26
27      // Save the dice values.
28      die1Value = die1.getValue();
29      die2Value = die2.getValue();
30  }
31
```

```
32  //*****************************************
33  // The getChoOrHan member function returns  *
34  // the result of the dice roll, Cho (even)  *
35  // or Han (odd).                            *
36  //*****************************************
37  string Dealer::getChoOrHan()
38  {
39      string result; // To hold the result
40
41      // Get the sum of the dice.
42      int sum = die1Value + die2Value;
43
44      // Determine even or odd.
45      if (sum % 2 == 0)
46          result = "Cho (even)";
47      else
48          result = "Han (odd)";
49
50      // Return the result.
51      return result;
52  }
53
54  //*****************************************
55  // The getDie1Value member function returns *
56  // the value of die #1.                     *
57  //*****************************************
58  int Dealer::getDie1Value()
59  {
60      return die1Value;
61  }
62
63  //*****************************************
64  // The getDie2Value member function returns *
65  // the value of die #2.                     *
66  //*****************************************
67  int Dealer::getDie2Value()
68  {
69      return die2Value;
70  }
```

Here is a synopsis of the class members:

`die1`	Declared in line 11 of Dealer.h. This is an instance of the `Die` class, to represent die #1.
`die2`	Declared in line 12 of Dealer.h. This is an instance of the `Die` class, to represent die #2.
`die1Value`	Declared in line 13 of Dealer.h. This member variable will hold the value of die #1 after it has been rolled.
`die2Value`	Declared in line 14 of Dealer.h. This member variable will hold the value of die #1 after it has been rolled.
Constructor	Lines 9 through 15 of Dealer.cpp initializes the `die1Value` and `die2Value` member variables to 0.

`rollDice` Lines 21 through 30 of Dealer.cpp. This member function simulates the rolling of the dice. Lines 24 and 25 call the `Die` objects' `roll` method. Lines 28 and 29 save the value of the dice in the `die1Value` and `die2Value` member variables.

`getChoOrHan` Lines 37 through 52 of Dealer.cpp. This member function returns a string indicating whether the sum of the dice is Cho (even) or Han (odd).

`getDie1Value` Lines 58 through 61 of Dealer.cpp. This member function returns the value of first die (stored in the `die1Value` member variable).

`getDie2Value` Lines 67 through 70 of Dealer.cpp. This member function returns the value of first die (stored in the `die2Value` member variable).

Now, let's look at the `Player` class.

Contents of `Player.h`

```
 1  // Specification file for the Player class
 2  #ifndef PLAYER_H
 3  #define PLAYER_H
 4  #include <string>
 5  using namespace std;
 6
 7  class Player
 8  {
 9  private:
10      string name;           // The player's name
11      string guess;          // The player's guess
12      int points;            // The player's points
13
14  public:
15      Player(string);        // Constructor
16      void makeGuess();      // Causes player to make a guess
17      void addPoints(int);   // Adds points to the player
18      string getName();      // Returns the player's name
19      string getGuess();     // Returns the player's guess
20      int getPoints();       // Returns the player's points
21  };
22  #endif
```

Contents of `Player.cpp`

```
 1  // Implementation file for the Player class
 2  #include "Player.h"
 3  #include <random>
 4  #include <string>
 5  using namespace std;
 6
 7  //*********************************************
 8  // Constructor                                *
 9  //*********************************************
10  Player::Player(string playerName)
```

```
11  {
12      name = playerName;
13      guess = "";
14      points = 0;
15  }
16
17  //*********************************************
18  // The makeGuess member function causes the    *
19  // player to make a guess, either "Cho (even)" *
20  // or "Han (odd)".                             *
21  //*********************************************
22  void Player::makeGuess()
23  {
24      const int MIN = 0;  // Mimimum random value
25      const int MAX = 1;  // Maximum random value
26      int guessNumber; // For the user's guess
27
28      // Define a random number engine and a distribution object.
29      random_device engine;
30      uniform_int_distribution<int> randomGuess(MIN, MAX);
31
32      // Get a random number, either 0 or 1.
33      guessNumber = randomGuess(engine);
34
35      // Convert the random number to Cho or Han.
36      if (guessNumber == 0)
37          guess = "Cho (even)";
38      else
39          guess = "Han (odd)";
40  }
41
42  //*********************************************
43  // The addPoints member function adds a        *
44  // specified number of points to the player's  *
45  // current balance.                            *
46  //*********************************************
47  void Player::addPoints(int newPoints)
48  {
49      points += newPoints;
50  }
51
52  //*********************************************
53  // The getName member function returns a       *
54  // player's name.                              *
55  //*********************************************
56  string Player::getName()
57  {
58      return name;
59  }
60
61  //*********************************************
62  // The getGuess member function returns a      *
63  // player's guess.                             *
64  //*********************************************
```

```
65  string Player::getGuess()
66  {
67      return guess;
68  }
69
70  //*********************************************
71  // The getPoints member function returns a    *
72  // player's points.                           *
73  //*********************************************
74  int Player::getPoints()
75  {
76      return points;
77  }
```

Here is a synopsis of the class members:

`name`	Declared in line 10 of Player.h. This member variable will hold the player's name.
`guess`	Declared in line 11 of Player.h. This member variable will hold the player's guess.
`points`	Declared in line 12 of Player.h. This member variable will hold the player's points.
Constructor	Lines 10 through 15 of Player.cpp, accepts an argument for the player's name, which is assigned to the `name` field. The `guess` field is assigned an empty string, and the `points` field is set to 0.
`makeGuess`	Lines 22 through 40 of Player.cpp. This member function causes the player to make a guess. The function generates a random number that is either a 0 or a 1. The `if` statement that begins at line 36 assigns the string `"Cho (even)"` to the `guess` member variable if the random number is 0, or it assigns the string `"Han (odd)"` to the `guess` member variable if the random number is 1.
`addPoints`	Lines 47 through 50 of Player.cpp. This member function adds the number of points specified by the argument to the player's `point` member variable.
`getName`	Lines 56 through 59 of Player.cpp. This member function returns the player's name.
`getGuess`	Lines 65 through 68 of Player.cpp. This member function returns the player's guess.
`getPoints`	Lines 74 through 77 of Player.cpp. This member function returns the player's points.

Program 14-17 uses these classes to simulate the game. The `main` function simulates five rounds of the game, displaying the results of each round, then displays the overall game results.

Program 14-17

```
1  // This program simulates the game of Cho-Han.
2  #include <iostream>
3  #include <string>
4  #include "Dealer.h"
5  #include "Player.h"
6  using namespace std;
7
```

(program continues)

Program 14-17 *(continued)*

```
// Function prototypes
void roundResults(Dealer &, Player &, Player &);
void checkGuess(Player &, Dealer &);
void displayGrandWinner(Player, Player);

int main()
{
    const int MAX_ROUNDS = 5;   // Number of rounds
    string player1Name;         // First player's name
    string player2Name;         // Second player's name

    // Get the player's names.
    cout << "Enter the first player's name: ";
    cin >> player1Name;
    cout << "Enter the second player's name: ";
    cin >> player2Name;

    // Create the dealer.
    Dealer dealer;

    // Create the two players.
    Player player1(player1Name);
    Player player2(player2Name);

    // Play the rounds.
    for (int round = 0; round < MAX_ROUNDS; round++)
    {
        cout << "----------------------------\n";
        cout << "Now playing round " << (round + 1)
             << endl;

        // Roll the dice.
        dealer.rollDice();

        // The players make their guesses.
        player1.makeGuess();
        player2.makeGuess();

        // Determine the winner of this round.
        roundResults(dealer, player1, player2);
    }

    // Display the grand winner.
    displayGrandWinner(player1, player2);
    return 0;
}

//*************************************************
// The roundResults function determines the results *
// of the current round.                            *
//*************************************************
void roundResults(Dealer &dealer, Player &player1, Player &player2)
```

```
60  {
61      // Show the dice values.
62      cout << "The dealer rolled " << dealer.getDie1Value()
63           << " and " << dealer.getDie2Value() << endl;
64
65      // Show the result (Cho or Han).
66      cout << "Result: " << dealer.getChoOrHan() << endl;
67
68      // Check each player's guess and award points.
69      checkGuess(player1, dealer);
70      checkGuess(player2, dealer);
71  }
72
73  //*************************************************
74  // The checkGuess function checks a player's guess*
75  // against the dealer's result.                   *
76  //*************************************************
77  void checkGuess(Player &player, Dealer &dealer)
78  {
79      const int POINTS_TO_ADD = 1; // Points to award winner
80
81      // Get the player's guess
82      string guess = player.getGuess();
83
84      // Get the result (Cho or Han).
85      string choHanResult = dealer.getChoOrHan();
86
87      // Display the player's guess.
88      cout << "The player " << player.getName()
89           << " guessed " << player.getGuess() << endl;
90
91      // Award points if the player guessed correctly.
92      if (guess == choHanResult)
93      {
94          player.addPoints(POINTS_TO_ADD);
95          cout << "Awarding " << POINTS_TO_ADD
96               << " point(s) to " << player.getName()
97               << endl;
98      }
99  }
100
101 //*************************************************
102 // The displayGrandWinner function displays the   *
103 // game's grand winner.                           *
104 //*************************************************
105 void displayGrandWinner(Player player1, Player player2)
106 {
107     cout << "---------------------------\n";
108     cout << "Game over. Here are the results:\n";
109
```

(program continues)

Program 14-17 *(continued)*

```
110      // Display player #1's results.
111      cout << player1.getName() << ": "
112           << player1.getPoints() << " points\n";
113
114      // Display player #2's results.
115      cout << player2.getName() << ": "
116           << player2.getPoints() << " points\n";
117
118      // Determine the grand winner.
119      if (player1.getPoints() > player2.getPoints())
120      {
121          cout << player1.getName()
122               << " is the grand winner!\n";
123      }
124      else if (player2.getPoints() > player1.getPoints())
125      {
126          cout << player2.getName()
127               << " is the grand winner!\n";
128      }
129      else
130      {
131          cout << "Both players are tied!\n";
132      }
133  }
```

Program Output with Example Input Shown in Bold

```
Enter the first player's name: Bill [Enter]
Enter the second player's name: Jill [Enter]
---------------------------
Now playing round 1
The dealer rolled 4 and 6
Result: Cho (even)
The player Bill guessed Cho (even)
Awarding 1 point(s) to Bill
The player Jill guessed Cho (even)
Awarding 1 point(s) to Jill
---------------------------
Now playing round 2
The dealer rolled 5 and 2
Result: Han (odd)
The player Bill guessed Han (odd)
Awarding 1 point(s) to Bill
The player Jill guessed Han (odd)
Awarding 1 point(s) to Jill
---------------------------
Now playing round 3
The dealer rolled 4 and 2
Result: Cho (even)
The player Bill guessed Cho (even)
Awarding 1 point(s) to Bill
```

```
The player Jill guessed Cho (even)
Awarding 1 point(s) to Jill
---------------------------
Now playing round 4
The dealer rolled 3 and 2
Result: Han (odd)
The player Bill guessed Han (odd)
Awarding 1 point(s) to Bill
The player Jill guessed Cho (even)
---------------------------
Now playing round 5
The dealer rolled 4 and 6
Result: Cho (even)
The player Bill guessed Han (odd)
The player Jill guessed Han (odd)
---------------------------
Game over. Here are the results:
Bill: 4 points
Jill: 3 points
Bill is the grand winner!
```

Let's look at the code. Here is a summary of the `main` function:

- Lines 15 through 17 make the following declarations: `MAX_ROUNDS`—the number of rounds to play, `player1Name`—to hold the name of player #1, and `player2Name`—to hold the name of player #2.
- Lines 20 through 23 prompt the user to enter the player's names.
- Line 26 creates an instance of the `Dealer` class to represent the dealer.
- Line 29 creates an instance of the `Player` class to represent player #1. Notice `player1Name` is passed as an argument to the constructor.
- Line 30 creates another instance of the `Player` class to represent player #2. Notice `player2Name` is passed as an argument to the constructor.
- The `for` loop that begins in line 33 iterates five times, causing the simulation of five rounds of the game. The loop performs the following actions:
  - Line 40 causes the dealer to roll the dice.
  - Line 43 causes player #1 to make a guess (Cho or Han).
  - Line 44 causes player #2 to make a guess (Cho or Han).
  - Line 47 passes the `dealer`, `player1`, and `player2` objects to the `roundResults` function. The function displays the results of this round.
- Line 51 passes the `player1` and `player2` objects to the `displayGrandWinner` function, which displays the grand winner of the game.

The `roundResults` function, which displays the results of a round, appears in lines 59 through 71. Here is a summary of the function:

- The function accepts references to the `dealer`, `player1`, and `player2` objects as arguments.
- The statement in lines 62 and 63 displays the value of the two dice.
- Line 66 calls the `dealer` object's `getChoOrHan` function to display the results, Cho or Han.
- Line 69 calls the `checkGuess` function, passing the `player1` and `dealer` objects as arguments. The `checkGuess` function compares a player's guess to the dealer's result (Cho or Han), and awards points to the player if the guess is correct.
- Line 70 calls the `checkGuess` function, passing the `player2` and `dealer` objects as arguments.

The `checkGuess` function, which compares a player's guess to the dealer's result, awarding points to the player for a correct guess, appears in lines 77 through 99. Here is a summary of the function:

- The function accepts references to a `Player` object and the `Dealer` object as arguments.
- Line 79 declares the constant `POINTS_TO_ADD`, set to the value 1, which is the number of points to add to the player's balance if the player's guess is correct.
- Line 82 assigns the player's guess to the `string` object `guess`.
- Line 85 assigns the dealer's results (Cho or Han) to the `string` object `choHanResult`.
- The statement in lines 88 and 89 displays the player's name and guess.
- The `if` statement in line 92 compares the player's guess to the dealer's result. If they match, then the player guessed correctly, and line 94 awards points to the player.

The `displayGrandWinner` function, which displays the grand winner of the game, appears in lines 105 through 133. Here is a summary of the function:

- The function accepts the `player1` and `player2` objects as arguments.
- The statements in lines 111 through 116 display both players' names and points.
- The `if-else-if` statement that begins in line 119 determines which of the two players has the highest score and displays that player's name as the grand winner. If both players have the same score, a tie is declared.

14.10 Rvalue References and Move Semantics

CONCEPT: A move operation transfers resources from a source object to a target object. A move operation is appropriate when the source object is temporary and is about to be destroyed.

C++ has always been regarded as a high-performance language. Typically, it is the language of choice for developers who need to write fast, efficient code. The C++ 11 standard introduced new features that make C++ even faster and more efficient. One of these new features is the ability to use *move semantics* in your classes. In certain situations, you can use move semantics to greatly improve the performance of assignment operations. You can also use move semantics to write a new type of constructor known as a move constructor. Before we can discuss move semantics, however, we must introduce another new topic: rvalue references.

Lvalues and Rvalues

During the execution of a program, numerous entities, such as variables, class objects, and so on, are created in memory. For example, look at the following `main` function:

```
1  int main()
2  {
3     int x = 0;
4
5     x = 12;
6     cout << x << endl;
7     return 0;
8  }
```

In this code, an int variable named x is created in line 3. Although the variable is created by the statement in line 3, it is meant to be accessible to other statements. In fact, any statement in the variable's scope can use the name x to access the variable. For example, line 5 assigns the value 12 to the x variable, and line 6 displays the contents of the x variable.

Some entities, such as the variable x in the previous example, are created by definition statements and meant to exist in memory long enough for other statements to access them. Other entities, however, are meant to exist only temporarily. For example, look at the following code fragment:

```
int x;
x = 2 * 6;
```

When the second statement executes, the system evaluates the expression on the right side of the = operator. In this case, it gets the value of 2 * 6 and stores the result, 12, as a temporary value. Then, it copies the temporary value to the x variable. After the statement executes, the temporary value is no longer needed, so the system discards it.

Values that are returned from functions are also stored as temporary values. When a function returns a value, you have to immediately do something with the value, or you will lose it. For example, look at the following code fragment:

```
1  int square(int a)
2  {
3     return a * a;
4  }
5
6  int main()
7  {
8     int x = 0;
9
10    x = square(5);
11    cout << x << endl;
12    return 0;
13 }
```

Consider the statement in line 10. We can summarize the statement by saying it calls the square function, and assigns the value that is returned from the square function to the variable x. But more specifically, all of the following things happen when this statement executes:

- The square function is called, and the value 5 is passed as an argument.
- The square function calculates 5 * 5 and stores the result, 25, as a temporary value.
- The temporary value is copied (assigned) to the variable x.
- The temporary value is no longer needed, so the system discards it.

To further illustrate this, look at the following code fragment:

```
1  int square(int a)
2  {
3     return a * a;
4  }
5
6  int main()
```

```
 7  {
 8     int x = 0;
 9
10     square(5);
11     cout << x << endl;
12     return 0;
13  }
```

In this code, the statement in line 10 calls the `square` function, but it doesn't do anything with the value that is returned. As in the previous example, the value 25 is stored as a temporary value. But, in this example, the value is lost forever after the statement has finished executing.

So, we have two types of values stored in memory during the execution of a program:

- Values that persist beyond the statement that created them, and have names that make them accessible to other statements in the program. In C++, these values are called *lvalues*.
- Values that are temporary, and cannot be accessed beyond the statement that created them. In C++, these values are called *rvalues*.



TIP: Perhaps the simplest way to think of an lvalue is this: it is an expression that can be written on the left side of an assignment operator.

Rvalue References

In Chapter 6, you learned that a reference variable is a variable that refers to the memory location of another variable. For example, look at the following code fragment:

```
int x = 34;
int &ref = x;
```

In this code, the variable `ref` is a reference variable. In a declaration, a reference is indicated by the presence of an ampersand (`&`) between the type and the variable's identifier. In C++ 11 and later, this type of reference is called an *lvalue reference* because it can refer only to lvalues.

C++ 11 also introduced the concept of an *rvalue reference*, which is a reference variable that can refer only to temporary objects that would otherwise have no name. An rvalue reference is declared similarly to an lvalue reference, except with a double ampersand (`&&`). The code below uses an rvalue reference to print the square of 5:

```
int &&rvalRef = square(5);
cout << rvalRef << endl;
```

The first statement calls the `square` function, passing 5 as an argument. The function returns the value 25, which is stored as a temporary value. The address of the temporary value is then assigned to `rvalRef`. So, now we have given a name to the temporary value, so it can be accessed by other statements.

When you assign an rvalue reference to a temporary value, you are giving a name to the temporary value and making it possible to access the value from other parts of the program. Therefore, you are transforming a temporary value into an lvalue. Because an rvalue reference cannot refer to an lvalue, the following code will not compile:

```
int x = 0;
int &&rvalRef = x;    // Error!
```

Look at the following initialization statement:

```
int &&rvalRef1 = square(5);
```

After this statement executes, the memory location containing the value returned by `square(5)` has a name, `rvalRef1`, so `rvalRef1` itself becomes an lvalue. This means that a subsequent initialization statement such as:

```
int &&rvalRef2 = rvalRef1;   // Error!
```

will not compile, because `rvalRef1` is no longer an rvalue. Fortunately, there is a positive consequence of all this: a temporary object can have at most one lvalue reference pointing to it. If a function has an lvalue reference to a temporary object, you can be sure no other part of the program has access to the same object. This fact is important in understanding how move operations, to be discussed next, work.

Move Semantics

When objects are passed as arguments into functions, and returned from functions, the program does a lot of work behind the scenes, allocating memory and copying values. To illustrate, let's look at the `Person` class.

```
#ifndef PERSON_H
#define PERSON_H
#include <cstring>
using namespace std;

class Person
{
private:
   char *name = nullptr;
public:
   // Default constructor
   Person()
   {  name = new char[1];
      *name = 0; }

   // Constructor with a parameter
   Person(char *n)
   {  name = new char[strlen(n) + 1];
      strcpy(name, n); }

   // Copy constructor
   Person(const Person &obj)
   {  name = new char[strlen(obj.name) + 1];
      strcpy(name, obj.name); }

   // Destructor
   ~Person()
   {  delete name; }
```

```
   // Copy assignment operator
   Person & operator=(const Person &right)
   {  if (this != &right)
      {
         name = new char[strlen(right.name) + 1];
         strcpy(name, right.name);
      }
      return *this; }

   // getName member function
   char *getName() const
   {  return name; }
};
#endif
```

The `Person` class has three constructors: a default constructor, a parameterized constructor, and a copy constructor. It also has an overloaded `=` operator, and a member function to get the `name` member variable. We are going to use this class to demonstrate some of the work that happens when an object is passed around in a program. But first, we are going to insert some `cout` statements so we can see when the constructors and the destructor are called, as well as the overloaded `=` operator. The modified class is shown here, followed by Program 14-18 which demonstrates the class.

Contents of `Person.h`

```
 1  #ifndef PERSON_H
 2  #define PERSON_H
 3  #include <iostream>
 4  #include <cstring>
 5  using namespace std;
 6
 7  class Person
 8  {
 9  private:
10     char *name;
11  public:
12     // Default constructor
13     Person()
14     {  cout << "*** default constructor ***\n";
15        name = nullptr; }
17
18     // Constructor with a parameter
19     Person(char *n)
20     {  cout << "*** parameterized constructor ***\n";
21        name = new char[strlen(n) + 1];
22        strcpy(name, n); }
23
24     // Copy constructor
25     Person(const Person &obj)
26     {  cout << "*** copy constructor ***\n";
27        name = new char[strlen(obj.name) + 1];
28        strcpy(name, obj.name); }
29
```

```
30      // Destructor
31      ~Person()
32      {  cout << "*** destructor ***\n";
33         delete [] name; }
34
35      // Overloaded = operator
36      Person & operator=(const Person &right)
37      {  cout << "*** assignment operator ***\n";
38         if (this != &right)
39         {
40            if (name != nullptr)
41               delete[] name;
42            name = new char[strlen(right.name) + 1];
43            strcpy(name, right.name);
44         }
45         return *this; }
46
47      // getName member function
48      char *getName() const
49      {  return name; }
50  };
51  #endif
```

Program 14-18

```
1   #include <iostream>
2   #include <string>
3   #include "Person.h"
4   using namespace std;
5
6   // Function prototype
7   Person makePerson();
8   void displayPerson(Person);
9
10  int main()
11  {
12      Person person;
13      person = makePerson();
14      displayPerson(person);
15      return 0;
16  }
17
18  Person makePerson()
19  {
20      Person p("Will MacKenzie");
21      return p;
22  }
23
24  void displayPerson(Person p2)
25  {
26      cout << p2.getName() << endl;
27  }
```

(program output continues)

Program 14-18 *(continued)*

Program Output

```
*** default constructor ***          (1) Object named person created in main
*** parameterized constructor ***    (2) Object named p created in makePerson
*** copy constructor ***             (3) Temporary object created as a copy of p
*** destructor ***                   (4) Object p destroyed
*** assignment operator ***          (5) Temporary object assigned to person in main
*** destructor ***                   (6) Temporary object destroyed
*** copy constructor ***             (7) p2 parameter object in displayPerson
Will MacKenzie                       (8) Displayed by the cout statement in line 26
*** destructor ***                   (9) p2 parameter object destroyed
*** destructor ***                  (10) person object in main destroyed
```

In the program's output, the comments that are shown to the right, in a different color, are not part of the output. We have added those so we can refer to specific lines in our discussion.

Let's step through the execution of the program, and take a closer look at its output:

- In the `main` function, the statement in line 12 creates a local `Person` object named `person`. This statement causes the `Person` class's default constructor to execute, displaying the output in line (1).
- Line 13 calls the `makePerson` function. Inside the `makePerson` function, the statement in line 20 creates a `Person` object named `p`, passing "Will MacKenzie" to the constructor. This causes the output in line (2) to be displayed.
- In line 21, the `makePerson` function returns the `Person` object named `p`. Before the function terminates, however, a temporary object is created, as a copy of the `p` object. This causes the temporary object's copy constructor to be called, displaying the output in line (3).
- As the `makePerson` function terminates, the `p` object is destroyed, causing the output in line (4) to be displayed.
- In line 13, the temporary object is assigned to the `person` object. This causes the `Person` class's overloaded `=` operator to be called, displaying the output in line (5).
- The temporary object is destroyed, causing the `Person` class's destructor to be called. This displays the output in line (6).
- Line 14 calls the `displayPerson` function, passing the `person` object as an argument.
- In line 24, the `p2` parameter object is created as a copy of the `person` object. This calls the copy constructor, and displays the output in line (7).
- Line 26 displays the output shown in line (8).
- The `displayPerson` function terminates, causing the `p2` parameter to be destroyed. This calls the `Person` class's destructor, displaying the output in line (9).
- In `main`, line 15 ends the program, causing the `person` object to be destroyed. The output in line (10) is displayed by the `Person` class's destructor.

We are especially interested in line 13:

```
person = makePerson();
```

that calls the `makePerson()` function:

```
Person makePerson()
{
   Person p("Will MacKenzie");
   return p;
}
```

The `makePerson()` function creates a local `Person` object named `p`. When the function returns `p`, a temporary object is created as a copy of `p`. This causes the `Person` class's copy constructor to execute. The copy constructor allocates a block of memory to hold the `name` array, then copies the `name` array from `p` to the temporary object.

Then in line 13, the overloaded assignment operator executes to copy the temporary object to the `person` object. The overloaded assignment operator also allocates a block of memory, and copies the `name` array from the temporary object to the `person` object. As you can see, the copy assignment goes to great lengths to avoid pointer sharing between the temporary object and the `person` object, but then right after that, the temporary object is destroyed and its `name` array is deleted.

With C++ 11 and later, we can add another assignment operator function, known as a *move assignment* operator, to the `Person` class. The idea behind the move assignment operator is to avoid all this work by simply swapping the members of the object being assigned with the temporary object. That way, when the temporary object is destroyed, its destructor deletes the memory that was previously owned by `person`, and `person` avoids having to copy the elements that were in the temporary object's `name` array.

The move assignment operator function for the `Person` class follows. To simplify the code, we have used the STL library function `swap` to swap the contents of two memory locations. The `swap` function is declared in the `<algorithm>` header file (you will learn more about the functions in the `<algorithm>` header file in Chapter 17).

```
// Move assignment operator
Person& operator=(Person&& right)
{  if (this != &right)
   {
      swap(name, right.name);
   }
   return *this;
}
```

Compare the class's move assignment operator to the class's copy assignment operator:

```
// Copy assignment operator
Person & operator=(const Person &right)
{  if (this != &right)
   {
      name = new char[strlen(right.name) + 1];
      strcpy(name, right.name);
   }
   return *this;
}
```

Notice the function header for the move assignment operator is different from the function header for the copy assignment operator in two ways:

- The move assignment operator takes an rvalue reference as parameter. This is because a move assignment should only be performed when the source of the assignment is a temporary object.
- The parameter in the move assignment operator is not `const`. This is because "moving" a resource from an object modifies it.

The move assignment, introduced in C++ 11, is much more efficient than copy assignment, and it should be used whenever the object being assigned from is temporary. There is also a *move constructor*, which should be used when creating a new object by initialization from a temporary object.

If you add a move assignment operator to a class, you should also add a *move constructor* to the class. A move constructor avoids unnecessary copying of data by "stealing" it from the temporary object. Here is the move constructor for the `Person` class. Again, note the parameter to the constructor is an rvalue reference, denoting that the parameter is a temporary object.

```
// Move constructor
Person(Person&& temp)
{   // Steal the name pointer from temp.
    name = temp.name;

    // Nullify the temp object's name pointer.
    temp.name = nullptr;
}
```

Note the parameter in the move constructor cannot be `const`. In addition, the temporary object that is the source for the move constructor must be left in a state that will allow its destructor to run without causing errors. That is why, in the `Person` class's move constructor, we set the temporary object's `name` pointer to `nullptr` before the function ends.

When to Implement Move Semantics in a Class

Anytime you write a class that contains a pointer or a reference to an outside piece of data, you should include a copy constructor, a move constructor, a copy assignment operator, and a move assignment operator. By adding move operations to such a class, you will ensure the best performance for your code.

At runtime, the move constructor and move assignment operator are automatically called when appropriate. Specifically, the compiler uses a move operation whenever an object is being assigned the value of a temporary object, or initialized with a copy of a temporary object.

Default Operations Provided by the Compiler

The motivation for writing copy constructors, move constructors, copy assignment operators, move assignment operators, and destructors is to make sure a class properly manages and disposes of resources, such as dynamically allocated memory. Writing

all of these operations for each class can be tedious, so the compiler tries to help out by generating default implementations for them. Let's assume we compile the following class:

```
class MyClass
{
private:
   int mydata = 0;
public:
   int getMyData()
   { return mydata; }
   void setMyData(int d)
   { mydata = d; }
};
```

The compiler will automatically generate the following:

- A default constructor `MyClass()`
- A copy constructor `MyClass(const MyClass &)`
- A copy assignment operator `MyClass & operator=(const MyClass &)`
- A move constructor `MyClass(MyClass &&)`
- A destructor `~MyClass()`

Keep in mind if you write your own version of *any* of these member functions, the compiler will not provide any of the default versions. So, if you write one of these member functions, you should write all of the others as well. For example, if you write a copy constructor for a class, then you should also write a move constructor, a copy assignment operator, a move assignment operator, and a destructor.

Alternatively, you can use the `default` keyword to declare which default operations you want the compiler to automatically provide. For example, look at the following class:

```
1  class MyClass
2  {
3  private:
4    int mydata = 0;
5
6  public:
7    // Constructors provided by default
8    MyClass() = default;
9    MyClass(const MyClass&) = default;
10   MyClass(MyClass&&) = default;
11
12   // Parameterized constructor
13   MyClass(int n)
14   { mydata = n; }
15
16   // Destructor
17   ~MyClass() = default;
18
19   // Copy assignment operator
```

```
20    MyClass& operator=(const MyClass&) = default;
21
22    // Accessor
23    int getMyData() const
24    { return mydata; }
25
26    // Mutator
27    void setMyData(int d)
28    { mydata = d; }
29 };
```

Notice the use of the `default` keyword in lines 8, 9, 10, 17, and 20. Here is a summary of each of those declarations:

- Line 8 causes the compiler to generate a default constructor.
- Line 9 causes the compiler to generate a default copy constructor.
- Line 10 causes the compiler to generate a default move constructor.
- Line 17 causes the compiler to generate a default destructor.
- Line 20 causes the compiler to generate a default copy assignment operator.

You can also use the `delete` keyword to declare which default operations you do not want the compiler to automatically provide. For example, if you want to make sure that objects of a particular class cannot be copied, you can explicitly tell the compiler to delete the copy constructor, move constructor, and copy assignment operator. Here is an example:

```
1  class MyClass
2  {
3  private:
4    int mydata = 0;
5
6  public:
7    // Constructors provided by default
8    MyClass() = default;
9    MyClass(const MyClass&) = delete;
10   MyClass(MyClass&&) = delete;
11
12   // Parameterized constructor
13   MyClass(int n)
14   { mydata = n; }
15
16   // Destructor
17   ~MyClass() = default;
18
19   // Deleted copy assignment operator
20   MyClass& operator=(const MyClass&) = delete;
21
22   // Accessor
```

```
23     int getMyData() const
24     { return mydata; }
25
26     // Mutator
27     void setMyData(int d)
28     { mydata = d; }
29  };
```

Notice the use of the `delete` keyword in lines 9, 10, and 20. Here is a summary of each of those declarations:

- Line 9 tells that the compiler that we do not want a default copy constructor.
- Line 10 tells that the compiler that we do not want a default move constructor.
- Line 20 tells that the compiler that we do not want a default copy assignment operator.

As a result of these declarations, objects of the `MyClass` class cannot be copied by a constructor or by an assignment operatior.

Review Questions and Exercises

Short Answer

1. Describe the difference between an instance member variable and a static member variable.
2. Assume a class named `Numbers` has the following static member function declaration:

   ```
   static void showTotal();
   ```

   Write a statement that calls the `showTotal` function.
3. A static member variable is declared in a class. Where is the static member variable defined?
4. What is a friend function?
5. Why is it not always a good idea to make an entire class a friend of another class?
6. What is memberwise assignment?
7. When is a copy constructor called?
8. How can the compiler determine if a constructor is a copy constructor?
9. Describe a situation where memberwise assignment is not desirable.
10. Why must the parameter of a copy constructor be a reference?
11. What is a default copy constructor?
12. Why would a programmer want to overload operators rather than use regular member functions to perform similar operations?
13. What is passed to the parameter of a class's `operator=` function?

14. Why shouldn't a class's overloaded = operator be implemented with a `void` operator function?
15. How does the compiler know whether an overloaded ++ operator should be used in prefix or postfix mode?
16. What is the `this` pointer?
17. What type of value should be returned from an overloaded relational operator function?
18. The class `Stuff` has both a copy constructor and an overloaded = operator. Assume `blob` and `clump` are both instances of the `Stuff` class. For each statement below, indicate whether the copy constructor or the overloaded = operator will be called:

```
Stuff blob = clump;
clump = blob;
blob.operator=(clump);
showValues(blob);     // blob is passed by value.
```

19. Explain the programming steps necessary to make a class's member variable static.
20. Explain the programming steps necessary to make a class's member function static.
21. Consider the following class declaration:

```
class Thing
{
private:
   int x;
   int y;
   static int z;
public:
   Thing()
      { x = y = z; }
   static void putThing(int a)
      { z = a; }
};
```

Assume a program containing the class declaration defines three `Thing` objects with the following statement:

```
Thing one, two, three;
```

How many separate instances of the `x` member exist?

How many separate instances of the `y` member exist?

How many separate instances of the `z` member exist?

What value will be stored in the `x` and `y` members of each object?

Write a statement that will call the `PutThing` member function *before* the objects above are defined.

22. Describe the difference between making a class a member of another class (object aggregation), and making a class a friend of another class.
23. What is the purpose of a forward declaration of a class?
24. Explain why memberwise assignment can cause problems with a class that contains a pointer member.
25. Why is a class's copy constructor called when an object of that class is passed by value into a function?

Fill-in-the-Blank

26. If a member variable is declared _____, all objects of that class have access to the same variable.
27. Static member variables are defined _____ the class.
28. A(n) _____ member function cannot access any nonstatic member variables in its own class.
29. A static member function may be called _____ any instances of its class are defined.
30. A(n) _____ function is not a member of a class, but has access to the private members of the class.
31. A(n) _____ tells the compiler that a specific class will be declared later in the program.
32. _____ is the default behavior when an object is assigned the value of another object of the same class.
33. A(n) _____ is a special constructor, called whenever a new object is initialized with another object's data.
34. _____ is a special built-in pointer that is automatically passed as a hidden argument to all nonstatic member functions.
35. An operator may be _____ to work with a specific class.
36. When overloading the _____ operator, its function must have a dummy parameter.
37. Making an instance of one class a member of another class is called _____.
38. Object aggregation is useful for creating a(n) _____ relationship between two classes.

Algorithm Workbench

39. Assume a class named `Bird` exists. Write the header for a member function that overloads the `=` operator for that class.
40. Assume a class named `Dollars` exists. Write the headers for member functions that overload the prefix and postfix `++` operators for that class.
41. Assume a class named `Yen` exists. Write the header for a member function that overloads the `<` operator for that class.
42. Assume a class named `Length` exists. Write the header for a member function that overloads cout's `<<` operator for that class.
43. Assume a class named `Collection` exists. Write the header for a member function that overloads the `[]` operator for that class.

True or False

44. T F Static member variables cannot be accessed by nonstatic member functions.
45. T F Static member variables are defined outside their class declaration.
46. T F A static member function may refer to nonstatic member variables of the same class, but only after an instance of the class has been defined.
47. T F When a function is declared a `friend` by a class, it becomes a member of that class.
48. T F A `friend` function has access to the private members of the class declaring it a `friend`.
49. T F An entire class may be declared a `friend` of another class.

50. T F In order for a function or class to become a friend of another class, it must be declared as such by the class granting it access.
51. T F If a class has a pointer as a member, it's a good idea to also have a copy constructor.
52. T F You cannot use the = operator to assign one object's values to another object, unless you overload the operator.
53. T F If a class doesn't have a copy constructor, the compiler generates a default copy constructor for it.
54. T F If a class has a copy constructor, and an object of that class is passed by value into a function, the function's parameter will *not* call its copy constructor.
55. T F The `this` pointer is passed to static member functions.
56. T F All functions that overload unary operators must have a dummy parameter.
57. T F For an object to perform automatic type conversion, an operator function must be written.
58. T F It is possible to have an instance of one class as a member of another class.

Find the Errors

Each of the following class declarations has errors. Locate as many as you can.

59.
```
class Box
{
   private:
      double width;
      double length;
      double height;
   public:
      Box(double w, l, h)
         { width = w; length = l; height = h; }
      Box(Box b) // Copy constructor
         { width = b.width;
           length = b.length;
           height = b.height; }

   ... Other member functions follow ...
};
```

60.
```
class Circle
{
   private:
      double diameter;
      int centerX;
      int centerY;
   public:
      Circle(double d, int x, int y)
         { diameter = d; centerX = x; centerY = y; }
      // Overloaded = operator
      void Circle=(Circle &right)
         { diameter = right.diameter;
           centerX = right.centerX;
           centerY = right.centerY; }

   ... Other member functions follow ...
};
```

61.
```
class Point
{
   private:
      int xCoord;
      int yCoord;
   public:
      Point (int x, int y)
         { xCoord = x; yCoord = y; }
      // Overloaded + operator
      void operator+(const &Point right)
         { xCoord += right.xCoord;
           yCoord += right.yCoord;
         }

   ... Other member functions follow ...
};
```

62.
```
class Box
{
   private:
      double width;
      double length;
      double height;
   public:
      Box(double w, l, h)
         { width = w; length = l; height = h; }
      // Overloaded prefix ++ operator
      void operator++()
         { ++width; ++length;}
      // Overloaded postfix ++ operator
      void operator++()
         { width++; length++;}

   ... Other member functions follow ...
};
```

63.
```
class Yard
{
   private:
      float length;
   public:
      yard(float l)
         { length = l; }
      // float conversion function
      void operator float()
         { return length; }

   ... Other member functions follow ...
};
```

Programming Challenges

1. **`Numbers` Class**

   Design a class `Numbers` that can be used to translate whole dollar amounts in the range 0 through 9999 into an English description of the number. For example, the number 713 would be translated into the string *seven hundred thirteen*, and 8203 would be translated into *eight thousand two hundred three*. The class should have a single integer member variable:

   ```
   int number;
   ```

   and a static array of `string` objects that specify how to translate key dollar amounts into the desired format. For example, you might use static strings such as

   ```
   string lessThan20[20] = {"zero", "one", ..., "eighteen", "nineteen"};
   string hundred = "hundred";
   string thousand = "thousand";
   ```

   The class should have a constructor that accepts a nonnegative integer and uses it to initialize the `Numbers` object. It should have a member function `print()` that prints the English description of the `Numbers` object. Demonstrate the class by writing a main program that asks the user to enter a number in the proper range then prints out its English description.

2. **Day of the Year**

   Assuming a year has 365 days, write a class named `DayOfYear` that takes an integer representing a day of the year and translates it to a string consisting of the month followed by day of the month. For example,

   Day 2 would be *January 2*.
   Day 32 would be *February 1*.
   Day 365 would be *December 31*.

   The constructor for the class should take as parameter an integer representing the day of the year, and the class should have a member function `print()` that prints the day in the month–day format. The class should have an integer member variable to represent the day, and should have static member variables holding `string` objects that can be used to assist in the translation from the integer format to the month–day format.

   Test your class by inputting various integers representing days and printing out their representation in the month–day format.

3. **Day of the Year Modification**

   Modify the `DayOfYear` class, written in Programming Challenge 2, to add a constructor that takes two parameters: a `string` object representing a month and an integer in the range 0 through 31 representing the day of the month. The constructor should then initialize the integer member of the class to represent the day specified by the month and day of month parameters. The constructor should terminate the program with an appropriate error message if the number entered for a day is outside the range of days for the month given.

Add the following overloaded operators:

`++` **prefix and postfix increment operators.** These operators should modify the `DayOfYear` object so it represents the next day. If the day is already the end of the year, the new value of the object will represent the first day of the year.

`--` **prefix and postfix decrement operators.** These operators should modify the `DayOfYear` object so it represents the previous day. If the day is already the first day of the year, the new value of the object will represent the last day of the year.

4. `NumDays` Class



VideoNote
Solving the NumDays Problem

Design a class called `NumDays`. The class's purpose is to store a value that represents a number of work hours and convert it to a number of days. For example, 8 hours would be converted to 1 day, 12 hours would be converted to 1.5 days, and 18 hours would be converted to 2.25 days. The class should have a constructor that accepts a number of hours, as well as member functions for storing and retrieving the hours and days. The class should also have the following overloaded operators:

`+` ***Addition operator.*** When two `NumDays` objects are added together, the overloaded `+` operator should return the sum of the two objects' hours members.

`-` ***Subtraction operator.*** When one `NumDays` object is subtracted from another, the overloaded `-` operator should return the difference of the two objects' hours members.

`++` ***Prefix and postfix increment operators.*** These operators should increment the number of hours stored in the object. When incremented, the number of days should be automatically recalculated.

`--` ***Prefix and postfix decrement operators.*** These operators should decrement the number of hours stored in the object. When decremented, the number of days should be automatically recalculated.

5. Time Off



NOTE: This assignment assumes you have already completed Programming Challenge 4.

Design a class named `TimeOff`. The purpose of the class is to track an employee's sick leave, vacation, and unpaid time off. It should have, as members, the following instances of the `NumDays` class described in Programming Challenge 4:

`maxSickDays`	A `NumDays` object that records the maximum number of days of sick leave the employee may take.
`sickTaken`	A `NumDays` object that records the number of days of sick leave the employee has already taken.
`maxVacation`	A `NumDays` object that records the maximum number of days of paid vacation the employee may take.
`vacTaken`	A `NumDays` object that records the number of days of paid vacation the employee has already taken.

`maxUnpaid`	A `NumDays` object that records the maximum number of days of unpaid vacation the employee may take.
`unpaidTaken`	A `NumDays` object that records the number of days of unpaid leave the employee has taken.

Additionally, the class should have members for holding the employee's name and identification number. It should have an appropriate constructor and member functions for storing and retrieving data in any of the member objects.

Input Validation: Company policy states that an employee may not accumulate more than 240 hours of paid vacation. The class should not allow the `maxVacation` *object to store a value greater than this amount.*

6. Personnel Report



NOTE: This assignment assumes you have already completed Programming Challenges 4 and 5.

Write a program that uses an instance of the `TimeOff` class you designed in Programming Challenge 5. The program should ask the user to enter the number of months an employee has worked for the company. It should then use the `TimeOff` object to calculate and display the employee's maximum number of sick leave and vacation days. Employees earn 12 hours of vacation leave and 8 hours of sick leave per month.

7. `Month` Class

Design a class named `Month`. The class should have the following private members:

`name`	A `string` object that holds the name of a month, such as "January," "February," and so on.
`monthNumber`	An integer variable that holds the number of the month. For example, January would be 1, February would be 2, and so on. Valid values for this variable are 1 through 12.

In addition, provide the following member functions:

- A default constructor that sets `monthNumber` to 1 and `name` to "January."
- A constructor that accepts the name of the month as an argument. It should set `name` to the value passed as the argument and set `monthNumber` to the correct value.
- A constructor that accepts the number of the month as an argument. It should set `monthNumber` to the value passed as the argument and set `name` to the correct month name.
- Appropriate set and get functions for the `name` and `monthNumber` member variables.
- Prefix and postfix overloaded `++` operator functions that increment `monthNumber` and set name to the name of next month. If `monthNumber` is set to 12 when these functions execute, they should set `monthNumber` to 1 and `name` to "January."
- Prefix and postfix overloaded `--` operator functions that decrement `monthNumber` and set `name` to the name of previous month. If `monthNumber` is set to 1 when these functions execute, they should set `monthNumber` to 12 and `name` to "December."

Also, you should overload `cout`'s `<<` operator and `cin`'s `>>` operator to work with the `Month` class. Demonstrate the class in a program.

8. **`Date` Class Modification**

   Modify the `Date` class in Programming Challenge 1 of Chapter 13. The new version should have the following overloaded operators:

   `++` ***Prefix and postfix increment operators.*** These operators should increment the object's `day` member.

   `--` ***Prefix and postfix decrement operators.*** These operators should decrement the object's `day` member.

   `-` ***Subtraction operator.*** If one `Date` object is subtracted from another, the operator should give the number of days between the two dates. For example, if April 10, 2014 is subtracted from April 18, 2014, the result will be 8.

   `<<` **`cout`*'s stream insertion operator.*** This operator should cause the date to be displayed in the form

   ```
   April 18, 2018
   ```

   `>>` **`cin`*'s stream extraction operator.*** This operator should prompt the user for a date to be stored in a `Date` object.

   The class should detect the following conditions and handle them accordingly:

   - When a date is set to the last day of the month and incremented, it should become the first day of the following month.
   - When a date is set to December 31 and incremented, it should become January 1 of the following year.
   - When a day is set to the first day of the month and decremented, it should become the last day of the previous month.
   - When a date is set to January 1 and decremented, it should become December 31 of the previous year.

   Demonstrate the class's capabilities in a simple program.

   Input Validation: The overloaded `>>` operator should not accept invalid dates. For example, the date 13/45/2018 should not be accepted.

9. **`FeetInches` Modification**

   Modify the `FeetInches` class discussed in this chapter so it overloads the following operators:

   ```
   <=
   >=
   !=
   ```

   Demonstrate the class's capabilities in a simple program.

10. **Corporate Sales**

    A corporation has six divisions, each responsible for sales to different geographic locations. Design a `DivSales` class that keeps sales data for a division, with the following members:

    - An array with four elements for holding four quarters of sales figures for the division.
    - A private static variable for holding the total corporate sales for all divisions for the entire year.

- A member function that takes four arguments, each assumed to be the sales for a quarter. The value of the arguments should be copied into the array that holds the sales data. The total of the four arguments should be added to the static variable that holds the total yearly corporate sales.
- A function that takes an integer argument within the range of 0–3. The argument is to be used as a subscript into the division quarterly sales array. The function should return the value of the array element with that subscript.

Write a program that creates an array of six `DivSales` objects. The program should ask the user to enter the sales for four quarters for each division. After the data are entered, the program should display a table showing the division sales for each quarter. The program should then display the total corporate sales for the year.

Input Validation: Only accept positive values for quarterly sales figures.

11. `FeetInches` Class Copy Constructor and `multiply` Function

Add a copy constructor to the `FeetInches` class. This constructor should accept a `FeetInches` object as an argument. The constructor should assign to the `feet` attribute the value in the argument's `feet` attribute, and assign to the `inches` attribute the value in the argument's `inches` attribute. As a result, the new object will be a copy of the argument object.

Next, add a `multiply` member function to the `FeetInches` class. The `multiply` function should accept a `FeetInches` object as an argument. The argument object's `feet` and `inches` attributes will be multiplied by the calling object's `feet` and `inches` attributes, and a `FeetInches` object containing the result will be returned.

12. `LandTract` Class

Make a `LandTract` class that is composed of two `FeetInches` objects: one for the tract's length, and one for the width. The class should have a member function that returns the tract's area. Demonstrate the class in a program that asks the user to enter the dimensions for two tracts of land. The program should display the area of each tract of land, and indicate whether the tracts are of equal size.

13. Carpet Calculator

The Westfield Carpet Company has asked you to write an application that calculates the price of carpeting for rectangular rooms. To calculate the price, you multiply the area of the floor (width times length) by the price per square foot of carpet. For example, the area of floor that is 12 feet long and 10 feet wide is 120 square feet. To cover that floor with carpet that costs $8 per square foot would cost $960. (12 × 10 × 8 = 960.)

First, you should create a class named `RoomDimension` that has two `FeetInches` objects as attributes: one for the length of the room, and one for the width. (You should use the version of the `FeetInches` class you created in Programming Challenge 11 with the addition of a `multiply` member function. You can use this function to calculate the area of the room.) The `RoomDimension` class should have a member function that returns the area of the room as a `FeetInches` object.

Next, you should create a `RoomCarpet` class that has a `RoomDimension` object as an attribute. It should also have an attribute for the cost of the carpet per square foot. The `RoomCarpet` class should have a member function that returns the total cost of the carpet.

Once you have written these classes, use them in an application that asks the user to enter the dimensions of a room and the price per square foot of the desired carpeting. The application should display the total cost of the carpet.

14. Parking Ticket Simulator

For this assignment, you will design a set of classes that work together to simulate a police officer issuing a parking ticket. The classes you should design are:

- **The `ParkedCar` Class:** This class should simulate a parked car. The class's responsibilities are:
  - To know the car's make, model, color, license number, and the number of minutes that the car has been parked
- **The `ParkingMeter` Class:** This class should simulate a parking meter. The class's only responsibility is:
  - To know the number of minutes of parking time that has been purchased
- **The `ParkingTicket` Class:** This class should simulate a parking ticket. The class's responsibilities are:
  - To report the make, model, color, and license number of the illegally parked car
  - To report the amount of the fine, which is $25 for the first hour or part of an hour that the car is illegally parked, plus $10 for every additional hour or part of an hour that the car is illegally parked
  - To report the name and badge number of the police officer issuing the ticket
- **The `PoliceOfficer` Class:** This class should simulate a police officer inspecting parked cars. The class's responsibilities are:
  - To know the police officer's name and badge number
  - To examine a `ParkedCar` object and a `ParkingMeter` object, and determine whether the car's time has expired
  - To issue a parking ticket (generate a `ParkingTicket` object) if the car's time has expired

Write a program that demonstrates how these classes collaborate.

15. Car Instrument Simulator

For this assignment you will design a set of classes that work together to simulate a car's fuel gauge and odometer. The classes you will design are:

- **The `FuelGauge` Class:** This class will simulate a fuel gauge. Its responsibilities are
  - To know the car's current amount of fuel, in gallons.
  - To report the car's current amount of fuel, in gallons.
  - To be able to increment the amount of fuel by 1 gallon. This simulates putting fuel in the car. (The car can hold a maximum of 15 gallons.)
  - To be able to decrement the amount of fuel by 1 gallon, if the amount of fuel is greater than 0 gallons. This simulates burning fuel as the car runs.
- **The `Odometer` Class:** This class will simulate the car's odometer. Its responsibilities are:
  - To know the car's current mileage.
  - To report the car's current mileage.

- To be able to increment the current mileage by 1 mile. The maximum mileage the odometer can store is 999,999 miles. When this amount is exceeded, the odometer resets the current mileage to 0.
- To be able to work with a `FuelGauge` object. It should decrease the `FuelGauge` object's current amount of fuel by 1 gallon for every 24 miles traveled. (The car's fuel economy is 24 miles per gallon.)

Demonstrate the classes by creating instances of each. Simulate filling the car up with fuel, then run a loop that increments the odometer until the car runs out of fuel. During each loop iteration, print the car's current mileage and amount of fuel.